🧠 AI Evolution Program

Parte 12 — Ingeniería de IA, MLOps, LLMOps y AgentOps

Industrializa datos, modelos y agentes mediante CI/CD, registros, serving, observabilidad, control de costos, recuperación y operación segura.

1. 148 — Ciclo de vida de datos, modelos y agentes
2. 149 — Experimentos, semillas y trazabilidad
3. 150 — Registro y promoción champion-challenger
4. 151 — CI/CD y pruebas para sistemas de IA
5. 152 — Serving online, batch y streaming
6. 153 — Observabilidad: logs, métricas y trazas
7. 154 — Deriva, feedback y evaluación continua
8. 155 — LLMOps y gestión de prompts
9. 156 — AgentOps y análisis de trayectorias
10. 157 — Costo, latencia, caching y capacidad
11. 158 — Resiliencia, idempotencia, rollback y recuperación
12. 159 — Proyecto: plataforma de IA observable

148 — Ciclo de vida de datos, modelos y agentes

Parte: 12 — Ingeniería de IA, MLOps, LLMOps y AgentOps
Nivel: experto · **Horas estimadas:** 6
Laboratorio: `workflow` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **ciclo de vida de datos, modelos y agentes** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar ciclo de vida de datos, modelos y agentes usando los conceptos `lifecycle`, `data`, `model`, `agent`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`lifecycle`, `data`, `model`, `agent`

🗺️ Ubicación en el mapa de la IA

Hasta la parte 11 construiste modelos y agentes como artefactos aislados; esta parte los convierte en **sistemas operados**. El ciclo de vida es el mapa base de todo MLOps: Sculley et al. (2015) mostraron que el código de ML es una fracción pequeña de un sistema real, y que la deuda técnica vive en los bordes — datos, configuración, serving, monitoreo. Esta clase define los tres ciclos (datos, modelos, agentes) que las once clases siguientes instrumentan, prueban, sirven, observan y recuperan.

📖 Fundamentos

🔄 Tres ciclos, no uno

Un sistema de IA en operación encadena **tres ciclos de vida** con ritmos y artefactos distintos:

1. **Ciclo de datos** — ingesta → validación → transformación → versionado → catálogo → expiración. Su artefacto es el *dataset versionado* (con esquema, estadísticas y hash). Cambia al ritmo del mundo: cada hora o cada día.

2. **Ciclo de modelos** — experimentación → entrenamiento → evaluación → registro → promoción → despliegue → monitoreo → reentrenamiento o retiro. Su artefacto es el *modelo registrado* (pesos + código + entorno + métricas). Cambia por decisión: días o semanas.
3. **Ciclo de agentes** — diseño de herramientas y prompts → evaluación de trayectorias → despliegue con límites → observación de intervenciones → ajuste de política. Su artefacto es la *configuración del agente* (prompt del sistema, herramientas, presupuestos, modelo base). Cambia con cada ajuste de prompt o herramienta: horas o días.

La diferencia clave: en software clásico el comportamiento lo define el **código**; en ML lo define **código + datos**; en agentes lo define **código + datos + modelo base + prompt + entorno de herramientas**. Cada término adicional es una fuente de cambio no controlado que el ciclo debe versionar y vigilar.

🧱 Etapas y artefactos con contrato

Cada transición entre etapas debe producir un artefacto **identificable y reproducible**:

```
etapa           artefacto                   identidad mínima
--------------  --------------------------  --------------------------------
ingesta         dataset crudo               fuente + fecha + hash contenido
validación      reporte de esquema          versión de reglas + resultado
entrenamiento   modelo candidato            dataset_id + código (commit) + semilla + hiperparámetros
evaluación      reporte de métricas         modelo_id + dataset_eval_id + métricas
registro        modelo versionado (v N)     todo lo anterior + firma de entrada/salida
despliegue      endpoint / job              modelo_id + configuración de serving
monitoreo       métricas de operación       ventana temporal + versión desplegada
```

Si no puedes responder «¿con qué datos, código y semilla se produjo este artefacto?», el ciclo está roto en ese punto: es el problema de **linaje** (lineage) que la clase 149 trata en detalle.

⚖️ Deuda técnica oculta (Sculley et al., 2015)

El paper canónico identifica deudas específicas de ML que motivan todo el ciclo:

- **Erosión de fronteras**: los modelos entrelazan señales (CACE: *Changing Anything Changes Everything*); no hay modularidad real entre features.
- **Dependencias de datos** más costosas que las de código: señales inestables, features legadas, cascadas de correcciones.
- **Bucles de retroalimentación**: el modelo influye en los datos futuros que lo reentrenan (directos e indirectos).
- **Antipatrones**: código pegamento, junglas de pipelines, configuración gigante sin pruebas.

La consecuencia operativa: el mantenimiento de un sistema de ML se parece más a operar una refinería (flujos, válvulas, sensores) que a mantener una biblioteca de funciones.

🤖 Qué añade el ciclo de agentes

Un agente añade tres fuentes de no determinismo que el ciclo debe encerrar: el **modelo base** (puede cambiar por versión del proveedor), el **entorno de herramientas** (una API externa cambia su respuesta) y la **trayectoria** (número variable de pasos). Por eso AgentOps (clase 156) versiona prompts y herramientas igual

que MLOps versiona datasets y pesos, y añade métricas propias: tasa de éxito de tarea, pasos por tarea, costo por tarea e intervenciones humanas.

🧮 Ejemplo trabajado

Un equipo opera un clasificador de churn. Reconstruyamos el linaje de un incidente: el lunes las predicciones se degradan. La tabla de artefactos dice:

```
artefacto          id / versión   producido con
dataset_train      ds-2025-12     ventana 2025-06→2025-11, hash 9f3a…
modelo             churn-v7       ds-2025-12 + commit a1b2c3 + seed 42, AUC 0.83
endpoint           prod           churn-v7 desde 2026-01-10
dataset_scoring    (diario)       hash del lunes: 77e1… — esquema OK
```

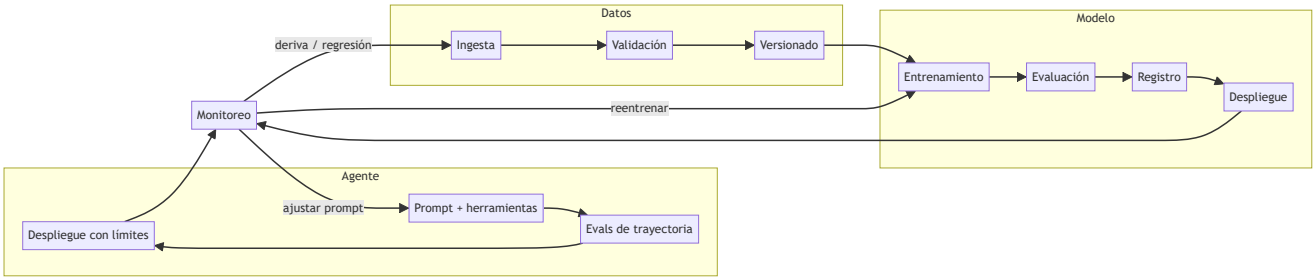
Diagnóstico por ciclos, en orden de velocidad de cambio:

1. **¿Ciclo de datos?** El esquema valida, pero la media de la feature `visitas_30d` pasó de 11.2 (entrenamiento) a 4.1 (lunes). Los datos del lunes provienen de una app que cambió su tracking el viernes. → deriva de datos (clase 154).
2. **¿Ciclo de modelo?** `churn-v7` no cambió (mismo id). Descartado como causa raíz.
3. **¿Ciclo de agente?** No aplica: no hay agente en este sistema.

Conclusión: la causa vive en el ciclo de datos; la corrección es reparar el tracking o reentrenar con la nueva distribución — y el hallazgo fue posible porque cada artefacto tenía identidad. Sin los hashes y las estadísticas de `ds-2025-12`, solo habría «el modelo anda mal», sin causa accionable.

📊 Propiedades y comparación

Dimensión	Ciclo de datos	Ciclo de modelos	Ciclo de agentes
Artefacto central	dataset versionado	modelo registrado	configuración (prompt + herramientas)
Ritmo de cambio	continuo (horas/días)	por decisión (días/semanas)	por ajuste (horas/días)
Fuente de fallo típica	deriva, esquema roto	sobreajuste, staleness	herramienta rota, prompt regresivo
Prueba clave	validación de esquema/estadísticas	evaluación vs. baseline	evals de trayectoria
Rollback	re-apuntar a versión anterior	re-desplegar versión N−1	restaurar prompt/config anterior
Quién dispara el cambio	el mundo	el equipo	el equipo y el proveedor del modelo



⚠️ Errores conceptuales frecuentes

1. **«MLOps = DevOps aplicado a modelos.»** DevOps versiona código; MLOps debe versionar además datos, features, modelos y configuración, y probar el comportamiento estadístico, no solo el funcional.
2. **«El ciclo termina en el despliegue.»** El despliegue es la mitad del ciclo: sin monitoreo y política de reentrenamiento/retiro, el modelo se degrada en silencio.
3. **«Un agente se opera igual que un modelo.»** El agente depende de un modelo base que puede cambiar sin tu intervención y de herramientas externas; su superficie de cambio es mayor y sus métricas (éxito de tarea, pasos, intervenciones) son distintas.
4. **«Versionar el código basta para reproducir.»** Sin el dataset exacto, la semilla y el entorno, el mismo commit produce otro modelo (clase 149).
5. **«La deuda técnica de ML se paga refactorizando código.»** Sculley et al. muestran que la mayor parte vive en dependencias de datos y configuración, invisibles al refactor clásico.

🚀 Del aprendizaje a la operación

El laboratorio simula un flujo con etapas y artefactos deterministas; una plataforma real añade orquestadores (Airflow, Kubeflow), stores de features, registro de modelos (MLflow), catálogos de datos con control de acceso, y acuerdos de nivel de servicio por etapa. La brecha principal no es técnica sino organizativa: definir quién es dueño de cada ciclo, quién aprueba una promoción y quién responde cuando el monitoreo dispara una alerta.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("workflow")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;

- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Sculley et al. (2015), "Hidden Technical Debt in Machine Learning Systems", NeurIPS — uso: referencia consultada en su fuente original
- Google Cloud, "MLOps: Continuous delivery and automation pipelines in machine learning" — uso: referencia consultada en su fuente original
- Google, "Rules of Machine Learning" (Martin Zinkevich) — uso: referencia consultada en su fuente original
- Paleyes, Urma & Lawrence (2022), "Challenges in Deploying Machine Learning", ACM Computing Surveys (arXiv:2011.09926) — uso: fuente primaria del mecanismo estudiado
- MLflow Documentation — uso: referencia consultada en su fuente original
- Huyen, *Designing Machine Learning Systems* (O'Reilly, 2022) — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P111 · Deuda técnica oculta en los sistemas de aprendizaje automático	2015	Nombra el hecho incómodo del área: el código del modelo es una fracción diminuta del sistema, y el resto acumula una deuda que ninguna herramienta detecta.	notebook
P115 · Hojas de datos para conjuntos de datos	2021	Traslada a los conjuntos de datos la hoja de características que acompaña a cualquier componente electrónico: qué es, cómo se hizo y para qué no sirve.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Huyen, Chip — *Designing Machine Learning Systems*	2022	ISBN 9781098107956 · web de la obra · *pendiente de confirmar en su catálogo*	citada en las referencias de esta clase · obra de referencia de la parte 12

📝 Evaluación completa

❓ Preguntas

1. Define ciclo de vida de datos, modelos y agentes sin usar una marca o framework como definición.
2. Explica la relación entre lifecycle, data, model, agent.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

149 — Experimentos, semillas y trazabilidad

Parte: 12 — Ingeniería de IA, MLOps, LLMOps y AgentOps
Nivel: experto · **Horas estimadas:** 6
Laboratorio: `observability` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **experimentos, semillas y trazabilidad** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar experimentos, semillas y trazabilidad usando los conceptos `experiments`, `seeds`, `lineage`, `artifacts`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`experiments`, `seeds`, `lineage`, `artifacts`

🗺️ Ubicación en el mapa de la IA

La ciencia experimental exige que un resultado sea reproducible; el ML industrial lo exige dos veces: para creer una métrica y para depurar producción. Esta clase toma el ciclo de vida de la clase 148 y lo hace **auditable**: tracking de experimentos, semillas, linaje de artefactos. Es el prerequisito del registro champion-challenger (150) y del CI/CD (151): no se puede promover ni probar lo que no se puede reproducir.

📖 Fundamentos

🎲 Fuentes de no determinismo

Un entrenamiento «igual» puede producir modelos distintos por:

1. **Semillas no fijadas**: inicialización de pesos, shuffling de batches, dropout, muestreo de negativos.
2. **Paralelismo**: reducciones en punto flotante no asociativas (`(a+b)+c ≠ a+(b+c)` en float32); kernels no deterministas de GPU (p. ej. atomics en cuDNN).
3. **Entorno**: versiones de librerías, drivers, hardware distinto.
4. **Datos**: el «mismo» dataset leído de una tabla viva que cambió entre corridas.

Fijar la semilla controla (1); (2) exige flags de determinismo (con costo de velocidad); (3) exige congelar el entorno (lockfiles, contenedores); (4) exige **snapshots versionados**. La reproducibilidad práctica se declara por niveles: *repetible* (misma máquina, mismos bits), *reproducible* (otro entorno, misma conclusión estadística), *replicable* (otros datos del mismo dominio, mismo hallazgo).

📋 Tracking de experimentos

Un experimento es una función `f(código, datos, configuración, azar) → métricas + artefactos`. El tracking registra **todas** las entradas y salidas para poder comparar y reproducir. El modelo mental de MLflow (equivalente en W&B o Neptune):

```
experimento  (agrupador: "churn-2026")
 └── run     (una ejecución)
      ├── params    : lr=0.05, depth=6, seed=42, dataset=ds-2025-12
      ├── metrics   : auc=0.831, logloss=0.412   (con historia por paso)
      ├── tags      : git_commit=a1b2c3, autor, propósito
      └── artifacts : modelo serializado, curvas, reporte de evaluación
```

Regla práctica: los `params` deben bastar para **relanzar** el run; las `metrics` deben bastar para **decidir** entre runs; los `artifacts` deben bastar para **desplegar** sin reentrenar.

🧬 Linaje (lineage)

El linaje responde «¿de qué proviene este artefacto?» como un grafo dirigido acíclico:

```
datos_crudos ──▶ ds-2025-12 ──▶ features_v3 ──▶ run_0042 ──▶ churn-v7
                      ▲                            ▲
                reglas_valid v2              commit a1b2c3 + seed 42
```

Cada nodo lleva una identidad estable — un **hash de contenido** (p. ej. SHA-256 del fichero o de las estadísticas del dataset) — y cada arista, la operación que lo produjo. Con esto, dos preguntas se vuelven consultas: *impacto* («si cambia `features_v3`, ¿qué modelos quedan obsoletos?», aristas hacia adelante) y *procedencia* («¿qué datos vio `churn-v7`?», aristas hacia atrás).

🔁 Protocolo mínimo de experimento honesto

1. Fija y registra la semilla **antes** de mirar resultados.
2. Congela el snapshot de datos y registra su hash.
3. Registra commit de código y entorno (versiones exactas).
4. Corre con ≥ 3 semillas si vas a comparar métodos: reporta media ± desviación, no el mejor run (eso es *seed picking*, una forma de sobreajuste al azar).
5. Decide el criterio de comparación antes de correr (métrica, dataset de evaluación, umbral de mejora mínima).

🧮 Ejemplo trabajado

Comparamos dos configuraciones de un clasificador con 3 semillas cada una (AUC en validación):

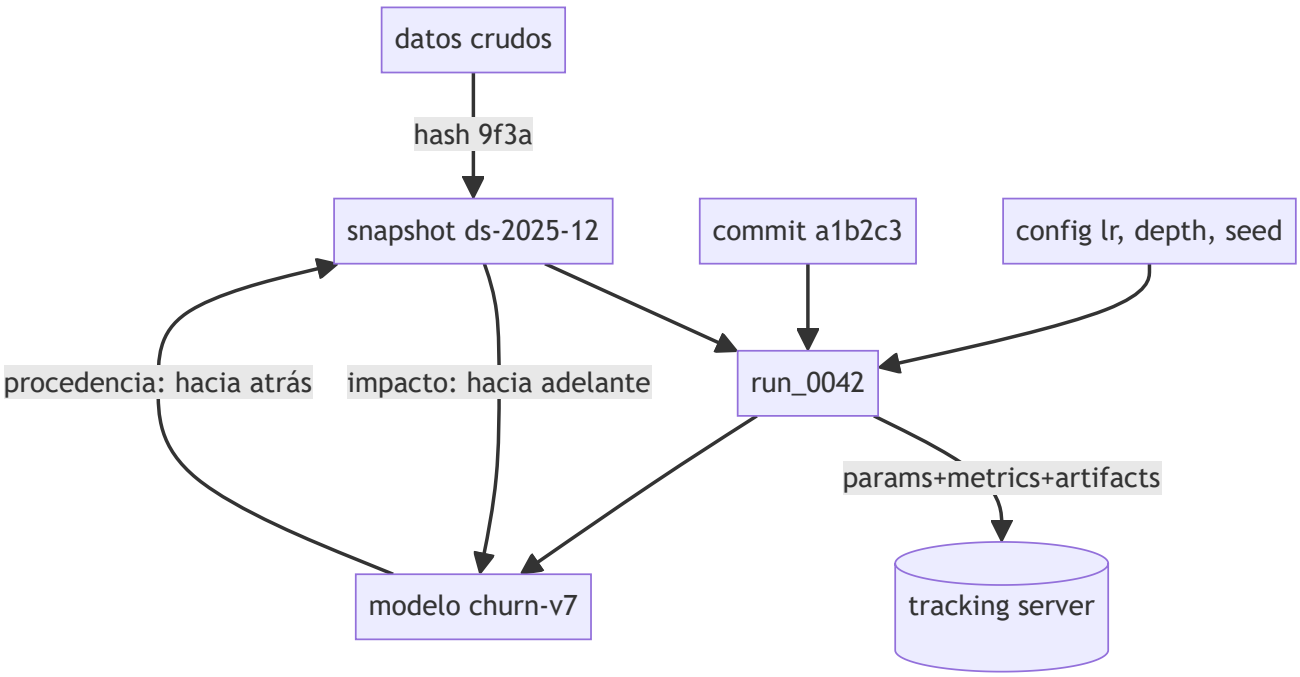
```
config A (lr=0.10): seeds 1,2,3 → 0.812, 0.804, 0.808   media 0.808  desv ≈ 0.004
config B (lr=0.05): seeds 1,2,3 → 0.815, 0.799, 0.802   media 0.805  desv ≈ 0.009
```

Lectura incorrecta: «B gana porque su mejor run (0.815) supera todo lo de A». Lectura correcta: la media de A (0.808) supera la de B (0.805), y la diferencia (0.003) es **menor que la variación entre semillas de B** (±0.009): con 3 semillas no hay evidencia para preferir ninguna; el «0.815» es ruido de semilla. Decisión honesta: o más semillas, o declarar empate y elegir por otro criterio (costo, simplicidad). Registro mínimo del run ganador si se promoviera A:

```
params : lr=0.10, seed={1,2,3}, dataset=ds-2025-12 (hash 9f3a…), commit a1b2c3
metrics: auc_media=0.808, auc_desv=0.004
```

📊 Propiedades y comparación

Práctica	Qué garantiza	Qué NO garantiza	Costo
Fijar semilla	repetibilidad del azar propio	determinismo de GPU/paralelismo	nulo
Flags deterministas	mismos bits en mismo hardware	velocidad (puede caer 10-30 %)	medio
Snapshot + hash de datos	mismas entradas	que los datos sean correctos	almacenamiento
Lockfile/contenedor	mismo entorno	mismo hardware numérico	mantenimiento
Multi-semilla + media±desv	conclusión robusta al azar	significancia con n=3	3-5× cómputo
Tracking (MLflow)	comparabilidad y auditoría	que registres lo importante	disciplina



⚠️ Errores conceptuales frecuentes

1. **«Fijé la semilla, luego es reproducible.»** La semilla no controla paralelismo, versiones de librerías ni datos vivos; es necesaria, no suficiente.
2. **«Reporto mi mejor run.»** Elegir la mejor semilla infla la métrica; lo comparable es media ± desviación sobre semillas preestablecidas.
3. **«El linaje es el historial de git.»** Git versiona código; el linaje une código con datos, configuración y artefactos. Un commit sin hash de dataset no reproduce nada.
4. **«Tracking = guardar métricas.»** Sin params, commit, entorno y artefactos, las métricas son incomparables e irreproducibles: números huérfanos.
5. **«Determinismo bit a bit siempre.»** A veces basta reproducibilidad estadística (misma conclusión); exigir bits idénticos en GPU puede costar rendimiento sin cambiar ninguna decisión.

🚀 Del aprendizaje a la operación

El laboratorio registra un flujo determinista con semilla explícita; una plataforma real añade un tracking server multiusuario (MLflow, W&B), versionado de datos a escala (DVC, lakeFS, Delta), convenciones de nombres obligatorias y retención de artefactos con costo de almacenamiento real. La disciplina que no se automatiza se pierde: los campos que el pipeline no registra automáticamente acaban vacíos, por eso el tracking se integra en el código de entrenamiento, no en la memoria del equipo.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("observability")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- MLflow Documentation — Tracking y conceptos — uso: referencia consultada en su fuente original
- Pineau et al. (2021), "Improving Reproducibility in Machine Learning Research", JMLR (arXiv:2003.12206) — uso: fuente primaria del mecanismo estudiado
- Google, "Rules of Machine Learning" — reglas sobre pipelines y métricas — uso: referencia consultada en su fuente original
- DVC Documentation — versionado de datos — uso: referencia consultada en su fuente original
- Sculley et al. (2015), "Hidden Technical Debt in Machine Learning Systems", NeurIPS — uso: referencia consultada en su fuente original
- PyTorch — Reproducibility notes — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P113 · Aprendizaje por refuerzo profundo que importa	2018	Demuestra empíricamente que con pocas semillas el ranking entre algoritmos es una moneda al aire, y que muchas mejoras publicadas no sobreviven a la comprobación.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Huyen, Chip — *Designing Machine Learning Systems*	2022	ISBN 9781098107956 · web de la obra · *pendiente de confirmar en su catálogo*	obra de referencia de la parte 12 · toda la parte

📝 Evaluación completa

❓ Preguntas

1. Define experimentos, semillas y trazabilidad sin usar una marca o framework como definición.
2. Explica la relación entre experiments, seeds, lineage, artifacts.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;

- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

150 — Registro y promoción champion-challenger

Parte: 12 — Ingeniería de IA, MLOps, LLMOps y AgentOps
Nivel: experto · **Horas estimadas:** 6
Laboratorio: `workflow` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **registro y promoción champion-challenger** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar registro y promoción champion-challenger usando los conceptos `registry`, `version`, `champion`, `challenger`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`registry`, `version`, `champion`, `challenger`

🗺️ Ubicación en el mapa de la IA

Con experimentos trazables (clase 149) el problema pasa a ser de gobierno: ¿qué versión del modelo sirve tráfico real y quién decide el reemplazo? El registro de modelos y el patrón **champion-challenger** trasladan a ML una idea vieja de la banca y el control de riesgo: la versión titular solo cede su puesto ante un retador que la supera bajo reglas acordadas de antemano. Es la antesala del CI/CD (151) y del serving (152).

📖 Fundamentos

🗂️ Registro de modelos

Un **registro** (model registry) es la fuente de verdad sobre qué modelos existen, en qué versión y en qué estado. A diferencia del tracking (que guarda *todos* los runs), el registro guarda los **candidatos con nombre y contrato**:

```
modelo registrado: "churn"
 ├── v6  — alias: (ninguno)     archivada
 ├── v7  — alias: champion      sirviendo producción
 └── v8  — alias: challenger    en evaluación en sombra
metadatos por versión: run de origen (linaje), firma de entrada/salida,
métricas de aceptación, aprobador, fecha de promoción
```

Los **estados/alias** forman una máquina de estados con transiciones explícitas: `registrada → en evaluación → champion → archivada`, y cada transición exige evidencia (métricas) y autorización (humana o automática con reglas). MLflow implementa esto con versiones y alias sobre un nombre de modelo.

🎈 Champion-challenger

- **Champion**: la versión que sirve tráfico y define el baseline vivo.
- **Challenger**: candidata que se evalúa contra el champion **con los mismos datos y las mismas métricas**, idealmente en *shadow mode*: recibe copia del tráfico real, sus predicciones se registran pero no se usan.

El patrón separa dos preguntas que suelen mezclarse: «¿el challenger es mejor?» (estadística) y «¿lo promovemos?» (decisión de negocio con costos asimétricos).

📏 Criterio de promoción

Una regla de promoción seria tiene cuatro componentes:

1. **Métrica primaria y umbral de mejora mínima** (Δ mínimo que paga el costo del cambio), evaluada con incertidumbre (varias semillas o intervalo por bootstrap).
2. **Guardas (guardrails)**: métricas que no pueden empeorar más de X aunque la primaria mejore — latencia p95, calibración, equidad por segmento, tasa de errores.
3. **Ventana y población de evaluación**: mismo periodo, mismo segmento; nunca comparar el champion de diciembre con el challenger de enero.
4. **Reversibilidad**: la promoción anterior queda archivada y re-desplegable (rollback en minutos, clase 158).

```
PROMOVER si:
  metrica_primaria(challenger) − metrica_primaria(champion) ≥ delta_min
  y para toda guarda g:  degradacion_g ≤ tolerancia_g
  y evaluacion hecha en la misma ventana/poblacion
EN OTRO CASO: mantener champion, archivar evidencia del challenger
```

🌓 Modos de evaluación en producción

- **Shadow**: challenger predice en paralelo sin afectar usuarios. Mide acuerdo con el champion y métricas técnicas; no mide impacto causal en negocio.
- **Canario / A-B**: challenger recibe un porcentaje pequeño de tráfico real. Mide impacto real; expone a usuarios y exige tamaño de muestra.
- **Replay offline**: evaluar el challenger sobre tráfico histórico etiquetado. Barato, pero ciego a bucles de retroalimentación y a features solo disponibles online.

🧮 Ejemplo trabajado

Modelo de riesgo crediticio. Regla acordada: promover si el AUC sube ≥ 0.005, con guardas: tasa de falsos negativos (FN) en el segmento «jóvenes sin historial» no puede subir más de 1 punto porcentual, y latencia p95 ≤ 150 ms.

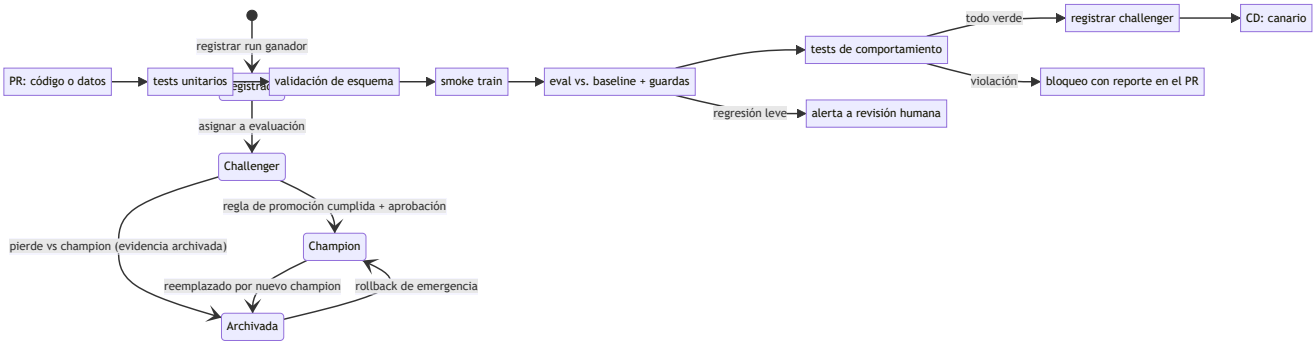
Evaluación en la misma ventana (4 semanas de tráfico en sombra, n = 48 000):

```
                     champion v7    challenger v8    Δ
AUC                  0.831          0.842            +0.011  ✓ (≥ 0.005)
FN jóvenes sin hist. 8.2 %          9.9 %            +1.7 pp ✗ (tolerancia 1.0)
latencia p95         110 ms         128 ms           +18 ms  ✓ (≤ 150)
```

Decisión: **NO promover**, aunque la métrica primaria mejora claramente. La guarda de equidad se viola: v8 gana AUC global empeorando justo el segmento protegido. Acciones posibles: reentrenar v8 con re-ponderación del segmento, o renegociar la guarda con quien es dueño del riesgo (decisión de negocio explícita, no default silencioso). El registro archiva la evaluación completa: dentro de seis meses alguien preguntará por qué v8 no se promovió, y la respuesta debe estar escrita.

📊 Propiedades y comparación

Modo de evaluación	Riesgo para usuarios	Mide impacto causal	Costo	Cuándo usar
Replay offline	nulo	no	bajo	primer filtro de candidatos
Shadow	nulo	no (solo acuerdo y métricas técnicas)	medio (2× cómputo)	validar contrato y estabilidad
Canario 1-5 %	bajo y acotado	sí, con muestra suficiente	medio	antes de promoción total
A/B formal	medio	sí, con potencia estadística	alto	cambios de alto impacto



⚠️ Errores conceptuales frecuentes

1. **«Gana la métrica primaria, se promueve.»** Sin guardas, se promueven modelos que mejoran el promedio degradando latencia, calibración o segmentos sensibles.
2. **«Comparar el AUC de validación del challenger con el AUC histórico del champion.»** Ventanas y poblaciones distintas no son comparables; la evaluación debe ser simultánea y sobre la misma población.
3. **«Shadow mode demuestra impacto en negocio.»** Solo demuestra estabilidad técnica y acuerdo; el impacto causal exige exponer tráfico (canario/A-B).
4. **«El registro es una carpeta de ficheros de pesos.»** Sin linaje, firma, estados y aprobadores, es almacenamiento, no gobierno: nadie sabe qué se puede desplegar.
5. **«Un delta positivo cualquiera justifica el cambio.»** Cambiar de modelo tiene costo (riesgo, revalidación, documentación); por eso existe `delta_min > 0`.

🚀 Del aprendizaje a la operación

El laboratorio decide una promoción con números deterministas; en producción se añade la incertidumbre muestral (intervalos, potencia estadística), aprobaciones humanas con segregación de funciones (quien entrena no aprueba), auditoría regulatoria en dominios como crédito o salud, y automatización del despliegue del nuevo champion con rollback listo. La regla de promoción escrita *antes* de ver los números es lo que separa un proceso de gobierno de una discusión post-hoc.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("workflow")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- MLflow Documentation — Model Registry — uso: referencia consultada en su fuente original
- Google Cloud, "MLOps: Continuous delivery and automation pipelines in machine learning" — uso: referencia consultada en su fuente original
- Huyen, *Designing Machine Learning Systems*, cap. de despliegue y evaluación online — uso: referencia consultada en su fuente original
- Kohavi, Tang & Xu, *Trustworthy Online Controlled Experiments* (Cambridge UP) — uso: referencia consultada en su fuente original
- Breck et al. (2017), "The ML Test Score", IEEE Big Data — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P114 · Tarjetas de modelo para el reporte de modelos	2019	Propone un documento corto y estandarizado que acompaña a cada modelo, con evaluación **desagregada** por subgrupo y usos fuera de alcance declarados.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Huyen, Chip — *Designing Machine Learning Systems*	2022	ISBN 9781098107956 · web de la obra · *pendiente de confirmar en su catálogo*	citada en las referencias de esta clase · obra de referencia de la parte 12

📝 Evaluación completa

❓ Preguntas

1. Define registro y promoción champion-challenger sin usar una marca o framework como definición.
2. Explica la relación entre registry, version, champion, challenger.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;

- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

151 — CI/CD y pruebas para sistemas de IA

Parte: 12 — Ingeniería de IA, MLOps, LLMOps y AgentOps
Nivel: experto · **Horas estimadas:** 6
Laboratorio: `evaluation` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **ci/cd y pruebas para sistemas de ia** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar ci/cd y pruebas para sistemas de ia usando los conceptos `CI`, `tests`, `validation`, `contracts`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`CI`, `tests`, `validation`, `contracts`

🗺️ Ubicación en el mapa de la IA

El CI/CD clásico automatizó la pregunta «¿este cambio de código rompe algo?». En sistemas de IA la pregunta se triplica: el cambio puede venir del código, de los **datos** o del **modelo**, y «romper» puede significar un test rojo o una degradación estadística silenciosa. Esta clase adapta la integración y entrega continuas al ciclo de vida de la clase 148, usando el registro de la 147 como puerta de salida hacia el serving (152).

📖 Fundamentos

🔺 Tres ejes de prueba

Siguiendo el *ML Test Score* de Breck et al. (2017), un sistema de ML se prueba en cuatro frentes; aquí los agrupamos en tres ejes más la infraestructura:

1. **Tests de datos**: el insumo cumple su contrato. - Esquema: columnas, tipos, rangos (`edad ∈ [0, 120]`), nulos permitidos. - Distribución: estadísticas dentro de bandas esperadas (media, cuantiles, cardinalidad de categóricas). - Legalidad/privacidad: features permitidas, sin fuga de PII.
2. **Tests de modelo**: el artefacto entrenado es apto. - Calidad mínima contra baseline fijo (¿supera al heurístico y a la versión anterior?). - Sin *training-serving skew*: la feature calculada offline coincide con

la online. - Reproducibilidad del entrenamiento (semilla + snapshot, clase 149).

3. **Tests de comportamiento**: el modelo hace lo correcto en casos con significado. - **Invariancia**: cambiar el nombre propio en un texto no debe cambiar el sentimiento. - **Direccionales**: subir el ingreso no debe subir la probabilidad de impago. - **Casos mínimos de funcionalidad**: ejemplos canónicos que jamás deben fallar (la «suite de humo» del modelo, cf. CheckList de Ribeiro et al.).

🔁 CI/CD y CT

El whitepaper de MLOps de Google distingue:

- **CI**: además de compilar y testear código, valida datos y produce artefactos de pipeline probados.
- **CD**: despliega no un binario sino un **pipeline completo** que a su vez entrena y sirve modelos.
- **CT (Continuous Training)**: el reentrenamiento automático disparado por calendario, llegada de datos o deriva — exclusivo de ML, no existe en DevOps clásico.

```
pipeline de CI para un cambio (código o datos):
1. tests unitarios de código           (segundos)
2. validación de esquema de datos      (segundos)
3. entrenamiento en muestra pequeña    (minutos)   ← smoke train
4. evaluación vs. baseline + guardas   (minutos)
5. tests de comportamiento             (minutos)
6. registro del candidato (challenger) (clase 150)
solo si 1-6 pasan → CD: despliegue canario → promoción
```

Herramientas como **CML** (Continuous Machine Learning) insertan en cada pull request un reporte con métricas y gráficas del candidato, convirtiendo la revisión de modelos en revisión de código: el diff incluye el delta de métricas.

🚦 Qué bloquea y qué alerta

No todo test debe bloquear el despliegue. Regla práctica: **bloquean** los contratos (esquema, firma del modelo, casos mínimos de funcionalidad, guardas duras); **alertan** las señales estadísticas ruidosas (pequeñas caídas de una métrica secundaria, cambios de distribución leves), que exigen juicio humano. Un pipeline donde todo bloquea acaba con overrides sistemáticos; uno donde todo alerta, con alertas ignoradas.

🧮 Ejemplo trabajado

Un PR cambia el binning de la feature `ingreso`. El pipeline corre:

```
paso                        resultado
tests unitarios             ✓ 214/214
esquema de datos            ✓ (mismas columnas y tipos)
smoke train (5 % datos)     ✓ entrena sin error, 92 s
eval vs. baseline           accuracy 0.902 → 0.897  (Δ −0.005; umbral bloqueo −0.01)
test direccional            ✗ p(impago) SUBE al subir ingreso en 3.1 % de los casos
                              (umbral: 1 %)
casos mínimos               ✓ 40/40
```

Veredicto: **bloqueado por el test direccional**, no por la métrica. La caída de accuracy (−0.005) está dentro de la tolerancia y solo habría alertado; pero el modelo ahora viola monotonicidad esperada en una

fracción de casos tres veces mayor que la tolerada — señal típica de un binning con bordes mal ordenados. El autor corrige los bordes del bin, el direccional baja a 0.4 % y el PR entra. Nótese lo que el ejemplo enseña: la métrica agregada era aceptable; el **comportamiento** no.

📊 Propiedades y comparación

Tipo de test	Detecta	No detecta	Cuándo corre	¿Bloquea?
Unitario de código	bugs de lógica	problemas de datos/modelo	cada commit	sí
Esquema de datos	columnas/tipos/rangos rotos	deriva de distribución	cada ingesta y cada PR	sí
Distribución de datos	cambios estadísticos	causas del cambio	cada ingesta	alerta
Eval vs. baseline	regresión de calidad global	fallos por segmento	cada candidato	sí (umbral)
Comportamiento (invariancia/direccional/mínimos)	fallos con significado	regresiones fuera de los casos escritos	cada candidato	sí
Skew offline/online	features inconsistentes	deriva temporal	pre-despliegue + muestreo	sí

⚠️ Errores conceptuales frecuentes

1. **«Si los tests de código pasan, el sistema está sano.»** El código puede ser perfecto y el modelo inútil: los datos y el comportamiento estadístico necesitan sus propios tests (ese es el punto del ML Test Score).
2. **«La métrica agregada cubre el comportamiento.»** Un modelo puede mantener accuracy y violar invariancias o monotonicidad en subconjuntos críticos, como en el ejemplo.
3. **«CT es un cron que reentrena.»** Sin validación automática post-entrenamiento y regla de promoción, un cron reentrenando es una fábrica de regresiones desatendida.
4. **«Todo test debe bloquear.»** Las señales estadísticas ruidosas como bloqueo duro producen overrides crónicos; se degradan a alerta con revisión humana.
5. **«El skew se prueba una vez.»** La paridad offline/online se erosiona con cada cambio de pipeline; se verifica en CI y se muestrea en producción continuamente.

🚀 Del aprendizaje a la operación

El laboratorio ejecuta una batería de validaciones deterministas; un CI/CD real añade runners con GPU y colas (el smoke train compite por recursos), cachés de datasets, gestión de secretos para fuentes de datos, reportes de CML en el PR, y la política de quién puede hacer override de un bloqueo y cómo queda auditado. La inversión con mejor retorno suele ser la más barata: validación de esquema en cada ingesta, que atrapa la mayoría de los incidentes reales antes de que toquen un modelo.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("evaluation")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Breck et al. (2017), "The ML Test Score: A Rubric for ML Production Readiness", IEEE Big Data — uso: referencia consultada en su fuente original
- Google Cloud, "MLOps: Continuous delivery and automation pipelines in machine learning" — uso: referencia consultada en su fuente original
- Ribeiro et al. (2020), "Beyond Accuracy: Behavioral Testing of NLP Models with CheckList", ACL (arXiv:2005.04118) — uso: fuente primaria del mecanismo estudiado
- CML — Continuous Machine Learning (iterative.ai) — uso: referencia consultada en su fuente original
- Great Expectations Documentation — validación de datos — uso: referencia consultada en su fuente original
- Fowler, "Continuous Integration" — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P112 · La puntuación de pruebas de ML: una rúbrica de preparación para producción	2017	Convierte «¿está listo para producción?» en una rúbrica de 28 pruebas concretas, puntuada por su categoría más débil.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Huyen, Chip — *Designing Machine Learning Systems*	2022	ISBN 9781098107956 · web de la obra · *pendiente de confirmar en su catálogo*	obra de referencia de la parte 12 · toda la parte

📝 Evaluación completa

❓ Preguntas

1. Define ci/cd y pruebas para sistemas de ia sin usar una marca o framework como definición.
2. Explica la relación entre CI, tests, validation, contracts.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.

- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

152 — Serving online, batch y streaming

Parte: 12 — Ingeniería de IA, MLOps, LLMOps y AgentOps
Nivel: experto · **Horas estimadas:** 6
Laboratorio: `observability` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **serving online, batch y streaming** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar serving online, batch y streaming usando los conceptos `serving`, `batch`, `streaming`, `API`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`serving`, `batch`, `streaming`, `API`

🗺️ Ubicación en el mapa de la IA

Un modelo registrado y probado (clases 150-151) todavía no sirve a nadie: falta decidir **cómo llega la predicción al consumidor**. Online, batch y streaming son los tres patrones de serving que gobiernan latencia, costo y frescura, y condicionan todo lo que sigue: qué se puede observar (153), qué deriva se detecta y cuándo (154) y cuánto cuesta operar (157). Elegir mal aquí es caro de revertir: cambia la arquitectura completa.

📖 Fundamentos

⚡ Online (síncrono, por petición)

El consumidor llama a un endpoint (REST/gRPC) y **espera** la respuesta.

- Optimiza **latencia**: el presupuesto se mide en percentiles (p50/p95/p99), no en promedio — el promedio esconde la cola, y la cola es lo que ve el usuario que espera (Dean & Barroso, *The Tail at Scale*).
- Exige features disponibles en milisegundos (feature store online, cachés).

- Escala con réplicas + balanceador; el costo corre aunque no haya tráfico (capacidad reservada) o paga arranques en frío (serverless).
- Para LLMs se añade una métrica propia: **TTFT** (time to first token) con streaming de tokens, que mejora la latencia percibida sin cambiar el throughput.

📦 Batch (asíncrono, por lote)

Un job periódico puntúa un conjunto grande y **materializa** los resultados (tabla, fichero) que el consumidor lee después.

- Optimiza **throughput y costo por predicción**: sin restricción de latencia por ítem, se usan lotes grandes, hardware spot y horarios valle.
- La **frescura** queda acotada por la periodicidad: un score diario tiene hasta 24 h de edad. Si la decisión tolera esa edad, batch es casi siempre lo más barato y simple.
- Fallo benigno: el job se relanza; nadie espera en línea.

🌊 Streaming (asíncrono, por evento)

El modelo consume un flujo de eventos (Kafka/PubSub) y emite predicciones o features al paso de los datos.

- Optimiza **frescura sin petición**: reacciona a eventos (fraude en la transacción, features "últimos 10 minutos") con latencias de sub-segundo a segundos.
- Coste de complejidad alto: exactly-once vs. at-least-once, ventanas, reprocesamiento, estado distribuido.
- Patrón híbrido común: features en streaming + inferencia online (el endpoint lee features frescas precalculadas).

📐 Latencia vs. throughput

Son objetivos en tensión: agrupar peticiones en lotes (batching dinámico) sube el throughput de la GPU pero añade espera a cada petición individual. La relación de Little (`concurrencia = tasa × latencia`) da el esqueleto del cálculo de capacidad:

```
throughput_max ≈ (réplicas × lote) / latencia_por_lote
p. ej. 4 réplicas, lote 8, 100 ms/lote → ≈ 320 peticiones/s
```

La decisión de patrón se reduce a tres preguntas: **¿quién espera?** (usuario en línea → online), **¿qué edad de predicción tolera la decisión?** (horas → batch), **¿la señal es un evento que debe reaccionar solo?** (→ streaming).

🧮 Ejemplo trabajado

Sistema de recomendaciones de un e-commerce con 2 M de usuarios, de los cuales ~80 000 visitan al día. Comparamos servir el top-N con batch diario vs. online:

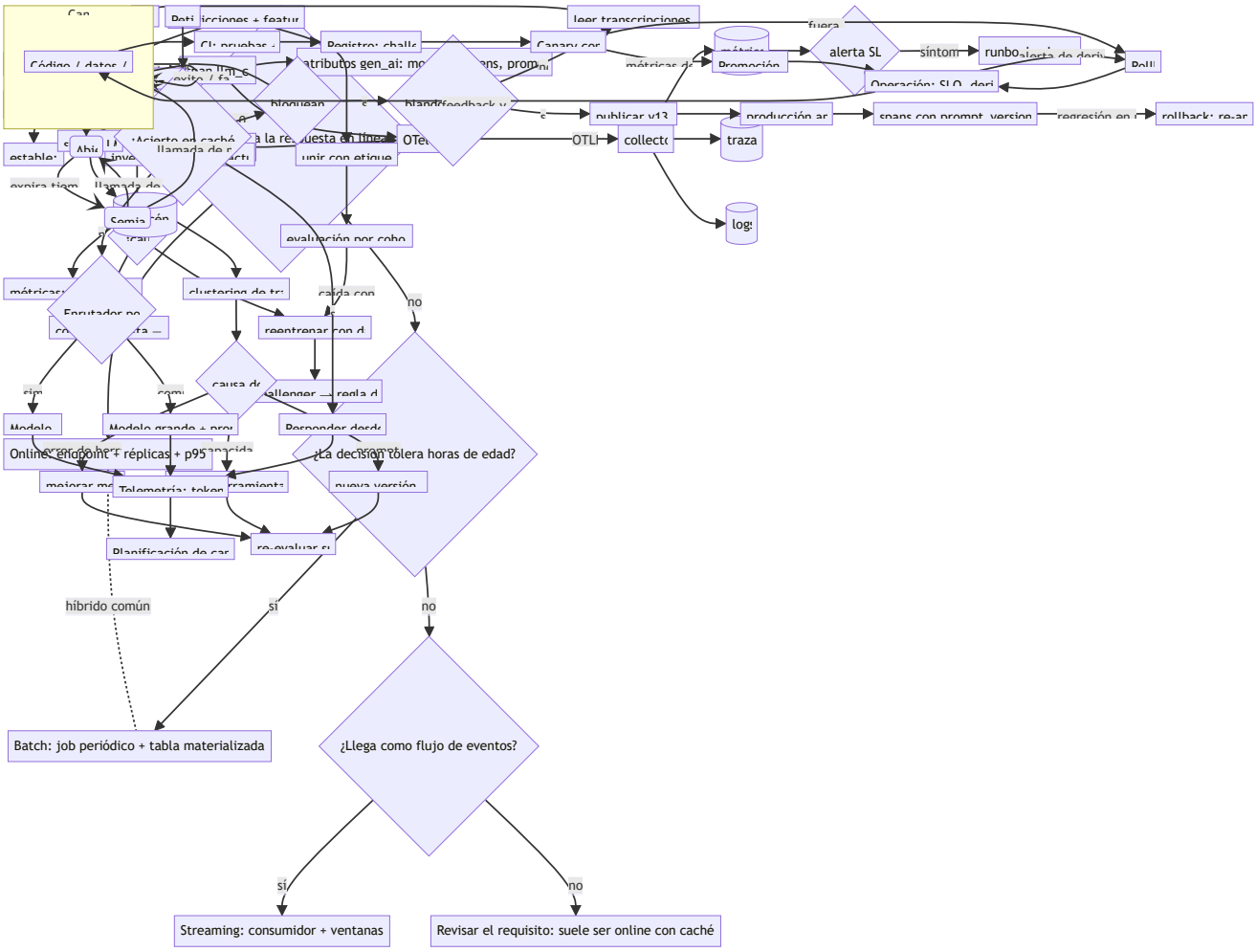
```
batch diario (todos los usuarios):
  2 000 000 usuarios × 1 scoring = 2.0 M predicciones/día
  utilidad: solo 80 000 se usan → 4 % de las predicciones se consumen
  frescura: hasta 24 h de edad; no reacciona a la sesión actual

online (solo visitantes):
  80 000 visitas × ~5 páginas = 400 000 predicciones/día  (5× menos cómputo útil total)
  pico: 80 000×5 / (8 h × 3600 s) ≈ 14 rps promedio; pico ~10× ≈ 140 rps
  con p95 requerido de 80 ms y ~50 rps por réplica → 3 réplicas + margen
  frescura: usa el carrito y los clics de la sesión actual
```

Decisión razonable: **híbrido** — candidatos pesados precalculados en batch (embeddings, top-500 por usuario, donde el 96 % de desperdicio es aceptable porque el costo unitario es bajo) y re-ranking online ligero con la señal de sesión. El ejemplo muestra el criterio general: batch para lo costoso y tolerante a edad, online para lo barato y sensible al contexto inmediato.

📊 Propiedades y comparación

Dimensión	Online	Batch	Streaming
Quién espera	el usuario (síncrono)	nadie (materializado)	nadie (reactivo)
Métrica reina	latencia p95/p99	costo por predicción y ventana de job	lag del consumidor y frescura
Frescura de features	ms (feature store online)	horas	segundos
Costo en reposo	réplicas encendidas	cero entre jobs	brokers + procesadores siempre activos
Complejidad operativa	media	baja	alta
Fallo típico	timeout, cola de latencia	job tardío o parcial	rezago (lag), duplicados
Ejemplo natural	scoring de una búsqueda	score de churn semanal	detección de fraude por transacción



⚠️ Errores conceptuales frecuentes

1. **«Optimizar la latencia promedio.»** El usuario del p99 existe y suele ser el que más peticiones hace; los SLO se definen sobre percentiles, y las colas se amplifican cuando una página hace N llamadas (la probabilidad de tocar el p99 crece con N).
2. **«Online es siempre mejor porque es más fresco.»** Si la decisión tolera horas de edad, online solo añade costo fijo, complejidad y modos de fallo nuevos.
3. **«Streaming = batch más rápido.»** Cambia el modelo de fallos completo: estado, reprocesamiento, garantías de entrega. No es un parámetro de velocidad, es otra arquitectura.
4. **«El throughput de la GPU es mi throughput.»** Sin batching dinámico, una GPU sirve peticiones de a una; el throughput real depende del tamaño de lote alcanzable dentro del presupuesto de latencia.
5. **«Puedo precalcular todo.»** Las señales de sesión (carrito actual, query actual) no existen antes de la visita; lo sensible al contexto inmediato es irreductiblemente online.

🚀 Del aprendizaje a la operación

El laboratorio contrasta los patrones con números deterministas; producción añade autoscaling con métricas de cola, feature stores con paridad offline/online, batching dinámico en el servidor de inferencia (Triton, vLLM para LLMs), colas de reintento y degradación elegante (responder con el resultado batch si el online

expira). El error de diseño más común no es técnico: es no haber escrito el requisito de frescura y el presupuesto de latencia **antes** de elegir el patrón.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("observability")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Dean & Barroso (2013), "The Tail at Scale", CACM — uso: referencia consultada en su fuente original
- Huyen, *Designing Machine Learning Systems* — cap. de serving y feature stores — uso: referencia consultada en su fuente original
- Kleppmann, *Designing Data-Intensive Applications* — batch y stream processing — uso: referencia consultada en su fuente original
- Google Cloud, "MLOps: Continuous delivery and automation pipelines in machine learning" — uso: referencia consultada en su fuente original
- Apache Kafka Documentation — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P109 · La cola a escala	2013	Muestra que con abanico grande la latencia de cola de cada componente se convierte en la latencia típica del sistema completo.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Huyen, Chip — *Designing Machine Learning Systems*	2022	ISBN 9781098107956 · web de la obra · *pendiente de confirmar en su catálogo*	citada en las referencias de esta clase · obra de referencia de la parte 12

📝 Evaluación completa

❓ Preguntas

1. Define serving online, batch y streaming sin usar una marca o framework como definición.
2. Explica la relación entre serving, batch, streaming, API.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

153 — Observabilidad: logs, métricas y trazas

Parte: 12 — Ingeniería de IA, MLOps, LLMOps y AgentOps
Nivel: experto · **Horas estimadas:** 6
Laboratorio: `observability` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **observabilidad: logs, métricas y trazas** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar observabilidad: logs, métricas y trazas usando los conceptos `logs`, `metrics`, `traces`, `OTel`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`logs`, `metrics`, `traces`, `OTel`

🗺️ Ubicación en el mapa de la IA

Servir un modelo (152) sin observarlo es volar sin instrumentos: los sistemas de IA fallan en silencio — la API responde 200 mientras la calidad se derrumba. Esta clase adopta el vocabulario estándar de la observabilidad (logs, métricas, trazas; OpenTelemetry como lingua franca) y lo extiende a lo que los sistemas clásicos no tienen: spans de inferencia LLM con tokens, costo y versión de prompt. Es la base sensorial de la deriva (154), del LLMOps (155) y del AgentOps (156).

📖 Fundamentos

🔭 Monitoreo vs. observabilidad

Monitorear es vigilar señales predefinidas («¿el p95 pasó de 200 ms?»). **Observabilidad** es la propiedad de un sistema cuyo estado interno puede inferirse desde sus salidas: permite responder preguntas que **no** se anticiparon («¿por qué justo los usuarios de iOS con prompts largos reciben respuestas truncadas?»). Se construye con tres señales complementarias:

📜 Logs — eventos discretos

Registros de hechos puntuales con contexto. La práctica moderna exige **logs estructurados** (JSON con campos, no texto libre) para poder consultarlos:

```
{"ts": "2026-07-30T10:02:11Z", "level": "WARN", "service": "inference",
 "trace_id": "a1b2…", "model": "churn-v7", "event": "fallback_used",
 "reason": "feature_store_timeout", "user_segment": "mobile"}
```

Fortaleza: máximo detalle por evento. Debilidad: volumen y costo; sin `trace_id` que los una, son anécdotas sueltas.

📈 Métricas — agregados numéricos

Series temporales baratas de almacenar y alertar. Tipos canónicos (Prometheus): **counter** (solo crece: `predicciones_total`), **gauge** (sube y baja: `replicas_activas`), **histogram** (distribución: `latencia_segundos_bucket`, del que se derivan percentiles). Para IA se añaden métricas de calidad y de costo: `proporcion_fallback`, `tokens_entrada/salida`, `costo_por_peticion`, `puntuacion_eval_muestreada`. Regla de cardinalidad: nunca usar identificadores de usuario o request como etiqueta — cada combinación de etiquetas crea una serie nueva.

🧵 Trazas — el viaje de una petición

Una **traza** es un árbol de **spans**; cada span es una operación con inicio, duración, atributos y estado, unidos por un `trace_id` que viaja por todos los servicios (propagación de contexto). OpenTelemetry estandariza API, SDK y el protocolo OTLP para exportar las tres señales; sus **convenciones semánticas para IA generativa** definen atributos como `gen_ai.request.model`, `gen_ai.usage.input_tokens`, `gen_ai.usage.output_tokens`, de modo que cualquier backend entienda un span de LLM.

```
traza: POST /answer                                  total 1 840 ms
 └─ span retrieve_context      (vector DB)             310 ms
 └─ span rerank                 (cross-encoder)         95 ms
 └─ span llm_call               (gen_ai.*)           1 380 ms
     atributos: model=claude-…, input_tokens=2 900,
                output_tokens=210, prompt_version=v12
 └─ span guardrail_check                              55 ms
```

Sin la traza solo verías «/answer tarda 1.8 s»; con ella sabes que el 75 % es la llamada al LLM y que el contexto recuperado mide 2 900 tokens — la palanca está en el retriever.

🚨 De señales a alertas

Alertar sobre **síntomas que duelen al usuario** (SLO de latencia, tasa de error, tasa de fallback), no sobre causas internas (CPU alta). Cada alerta debe ser accionable y llevar a un runbook; las señales de calidad de IA (score de evals muestreadas, deriva) suelen alertar a revisión humana, no paginar a las 3 a.m.

🚪 El gateway de IA: observabilidad y gobernanza en un solo punto

En la empresa, las tres señales convergen en un patrón arquitectónico que maduró en 2025-2026: el **AI gateway** — un proxy único por el que pasan todas las llamadas a modelos y agentes. Al concentrar el tráfico, el gateway obtiene gratis lo que costaría instrumentar servicio a servicio: trazas y costos por equipo/modelo/agente, límites de tasa y presupuesto, y enrutamiento con fallback entre proveedores. Sobre

esa base se apilan capas de gobernanza que ya son productos estándar: **registros** de qué modelos, APIs y skills están autorizados (descubrimiento incluido: qué IA está usando la organización *sin* permiso — shadow AI), y **agent personas** — la identidad operativa de cada agente que ata su descripción de rol a su modelo, herramientas y guardrails concretos, de modo que "qué puede hacer este agente" sea una configuración auditable y no una promesa del prompt. Es el mismo principio de mínimo privilegio de la clase 119, elevado de un agente a la flota completa.

🧮 Ejemplo trabajado

Presupuesto de latencia de un endpoint RAG con SLO p95 ≤ 2 000 ms. Medición de una ventana de 10 000 peticiones (histogramas por span):

```
span                p50      p95      contribución al p95 total
retrieve_context    180 ms   520 ms   26 %
rerank               60 ms   110 ms    6 %
llm_call            900 ms  1 450 ms  72 %
guardrail_check      40 ms    70 ms    4 %
p95 total observado: 1 980 ms  (los p95 no se suman: la traza muestra qué coincide)
```

El SLO está a 20 ms de romperse. Las trazas de las peticiones > 2 000 ms muestran un patrón: `input_tokens > 4 000` por contextos largos del retriever. Acción con datos: cap de contexto a 3 000 tokens → el p95 de `llm_call` cae a 1 150 ms y el total a 1 630 ms. La observabilidad convirtió «a veces va lento» en una decisión de ingeniería con una palanca concreta y verificable tras el cambio.

📊 Propiedades y comparación

Señal	Granularidad	Costo relativo	Responde	Límite
Logs estructurados	evento	alto (volumen)	¿qué pasó exactamente aquí?	correlación manual sin trace_id
Métricas	agregado	bajo	¿cuánto, cuántos, qué tan rápido?	sin detalle por petición; cardinalidad
Trazas	petición completa	medio (se muestrea)	¿dónde se fue el tiempo / qué llamó a qué?	muestreo pierde casos raros salvo tail-sampling
Spans gen_ai (OTel)	llamada LLM	medio	¿tokens, modelo, prompt, costo por llamada?	la calidad semántica exige evals aparte

⚠️ Errores conceptuales frecuentes

1. **«HTTP 200 = sistema sano.»** Un LLM puede responder 200 con alucinaciones o un modelo con scores degradados; la salud de IA exige señales de calidad muestreadas, no solo disponibilidad.
2. **«Los p95 de los spans se suman al p95 total.»** Los percentiles no son aditivos; solo la traza muestra cómo se componen las duraciones en cada petición real.
3. **«Logueo todo y ya soy observable.»** Logs sin estructura ni `trace_id` son texto caro; observabilidad es poder *consultar* lo no anticipado.
4. **«Etiqueto la métrica con el user_id para poder filtrar.»** Explosión de cardinalidad: miles de series nuevas; el detalle por usuario pertenece a logs/trazas.
5. **«Alerto sobre CPU y memoria.»** Son causas, no síntomas; se alerta sobre lo que el usuario sufre (SLO) y las causas se consultan al diagnosticar.

🚀 Del aprendizaje a la operación

El laboratorio emite señales deterministas; producción añade un collector de OTel con muestreo de colas (tail-based sampling para retener las trazas lentas), retención y costo por GB de logs, dashboards y SLOs con presupuesto de error, y redacción de PII antes de exportar prompts a un backend de trazas — un span gen_ai con el prompt completo es también un riesgo de privacidad. La regla de madurez: cada incidente que costó diagnosticar debe dejar tras de sí la señal que lo habría hecho trivial.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("observability")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- OpenTelemetry Documentation — uso: marco normativo de referencia
- OpenTelemetry — Semantic Conventions for Generative AI — uso: marco normativo de referencia
- Prometheus Documentation — tipos de métricas — uso: referencia consultada en su fuente original
- Google SRE Book — Monitoring Distributed Systems — uso: referencia consultada en su fuente original
- Dean & Barroso (2013), "The Tail at Scale", CACM — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P107 · Dapper, una infraestructura de trazado de sistemas distribuidos a gran escala	2010	Hace observable una petición que atraviesa decenas de servicios, con un identificador que viaja con ella y un muestreo que la hace asequible.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Huyen, Chip — *Designing Machine Learning Systems*	2022	ISBN 9781098107956 · web de la obra · *pendiente de confirmar en su catálogo*	obra de referencia de la parte 12 · toda la parte

Normas y documentación oficial que aplica esta clase: OpenTelemetry Documentation · OpenTelemetry

📝 Evaluación completa

❓ Preguntas

1. Define observabilidad: logs, métricas y trazas sin usar una marca o framework como definición.
2. Explica la relación entre logs, metrics, traces, OTel.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;

- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

154 — Deriva, feedback y evaluación continua

Parte: 12 — Ingeniería de IA, MLOps, LLMOps y AgentOps
Nivel: experto · **Horas estimadas:** 6
Laboratorio: `evaluation` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **deriva, feedback y evaluación continua** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar deriva, feedback y evaluación continua usando los conceptos `drift`, `feedback`, `continuous eval`, `monitoring`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`drift`, `feedback`, `continuous eval`, `monitoring`

🗺️ Ubicación en el mapa de la IA

Un modelo se entrena sobre una fotografía del mundo, y el mundo sigue moviéndose: la **deriva** es la razón por la que todo sistema de ML se degrada por defecto. Esta clase conecta la observabilidad (153) con la acción: detectar el cambio (PSI, monitoreo por segmento), cerrar el bucle de feedback con etiquetas reales y evaluar continuamente. Es el disparador del reentrenamiento (CT, clase 151) y de la decisión champion-challenger (150).

📖 Fundamentos

🌪️ Taxonomía de la deriva

Con entrada `X`, objetivo `Y` y modelo que aproxima `P(Y|X)`:

- **Deriva de datos (covariate shift)**: cambia `P(X)`; la relación `P(Y|X)` se mantiene. Ej.: llegan más usuarios jóvenes; el modelo acierta en cada segmento, pero opera fuera de su zona densa de entrenamiento.

- **Deriva de etiquetas (prior shift)**: cambia `P(Y)`. Ej.: la tasa base de fraude sube del 0.3 % al 1 % — la calibración del umbral queda obsoleta.
- **Deriva de concepto (concept drift)**: cambia `P(Y|X)` — la *regla del mundo*. Ej.: tras una crisis, el mismo perfil de cliente ahora sí impaga. Es la más grave: el modelo está objetivamente equivocado aunque las entradas parezcan normales.

La forma temporal importa: súbita (cambio de app), gradual (hábitos), recurrente (estacionalidad — que NO es deriva a corregir sino patrón a modelar).

📐 PSI: Population Stability Index

Métrica clásica (scoring bancario) para comparar la distribución esperada `p` (baseline, p. ej. entrenamiento) con la actual `q` sobre `B` bins:

```
PSI = Σ_{i=1..B} (q_i − p_i) · ln(q_i / p_i)
```

Cada término es ≥ 0 (si `q_i > p_i`, ambos factores positivos; si `q_i < p_i`, ambos negativos), así que PSI ≥ 0 y solo es 0 con distribuciones idénticas por bin. Es simétrica y equivale a la suma de las divergencias KL en ambos sentidos. Convención de la industria (regla empírica, no teorema): **< 0.10** estable; **0.10–0.25** cambio moderado, investigar; **> 0.25** cambio mayor, actuar. Precauciones: elegir bins sobre el baseline (deciles típicos), suavizar bins vacíos (q_i = 0 rompe el logaritmo), y recordar que el PSI depende del binning y del tamaño muestral.

🔁 Feedback y evaluación continua

Detectar deriva de `X` es fácil; saber si el **desempeño** cayó exige etiquetas reales, que llegan con retardo (el impago se conoce a 90 días) o sesgadas (solo ves el resultado de los créditos que aprobaste — *selective labels*). El bucle honesto:

1. **Proxies inmediatos**: tasa de fallback, distribución de scores, acuerdo con un modelo de referencia, quejas de usuarios.
2. **Etiquetas diferidas**: unir predicciones con resultados cuando maduran (evaluación retrospectiva por cohortes).
3. **Muestreo etiquetado**: presupuesto fijo de revisión humana continua (o LLM-judge calibrado con humanos) sobre una muestra aleatoria — no solo sobre los casos raros.
4. **Política de acción escrita**: qué PSI o qué caída de métrica dispara alerta, revisión, reentrenamiento o rollback. Detección sin política es un dashboard que nadie mira.

🧩 Dónde medir

Deriva por feature (PSI/KS por columna), deriva del score (distribución de salidas del modelo — barata y sorprendentemente sensible), y desempeño por **segmento** (una métrica global estable puede esconder un segmento en caída, cf. clase 150).

🧮 Ejemplo trabajado

Feature `monto_compra` binned en cuartiles del baseline de entrenamiento (p = 25 % cada bin, por construcción). Distribución del último mes:

```
bin          p (train)   q (mes)    (q-p)      ln(q/p)     término
Q1 bajo      0.25        0.15       −0.10      ln(0.60)=−0.511   0.0511
Q2           0.25        0.20       −0.05      ln(0.80)=−0.223   0.0112
Q3           0.25        0.30       +0.05      ln(1.20)=+0.182   0.0091
Q4 alto      0.25        0.35       +0.10      ln(1.40)=+0.336   0.0336
                                                        PSI  ≈  0.105
```

Lectura: PSI ≈ 0.105 cae en la banda 0.10–0.25 → **cambio moderado: investigar**. La masa migró hacia montos altos (Q4: 25 %→35 %). Investigación: ¿inflación?, ¿campaña de productos premium?, ¿bug que duplica montos? La acción depende de la causa: si es un cambio real y persistente del mundo, reentrenar con datos recientes; si es un bug de ingesta, corregirlo (reentrenar aprendería el error). El PSI **detecta**, no diagnostica: convierte «algo cambió» en «esto cambió, así, en esta feature».

📊 Propiedades y comparación

Método	Tipo de dato	Detecta	Necesita etiquetas	Nota
PSI	numérico binned / categórico	cambio de distribución	no	estándar bancario; umbrales 0.10/0.25
Test KS	numérico continuo	diferencia de CDFs	no	con n grande, significativo ante cambios triviales
Divergencia JS	distribuciones	cambio simétrico y acotado [0, ln2]	no	robusta a bins vacíos
Deriva del score	salidas del modelo	efecto agregado de cambios en X	no	primera alarma barata
Métrica con etiquetas diferidas	pred + resultado	caída real de desempeño	sí (con retardo)	la única verdad de fondo
Evals muestreadas (humano/juez)	casos individuales	degradación cualitativa	sí (muestra)	clave en LLMs sin etiqueta natural

⚠️ Errores conceptuales frecuentes

1. **«Detecté deriva de X, el modelo está roto.»** Covariate shift no implica caída de desempeño: la regla `P(Y|X)` puede seguir válida. La deriva de datos es una alarma temprana, no un veredicto.
2. **«Sin deriva de X, el modelo está bien.»** El concept drift cambia `P(Y|X)` sin mover necesariamente `P(X)`: entradas idénticas, mundo distinto. Solo las etiquetas lo confirman.

3. **«Reentrenar arregla toda deriva.»** Si la causa es un bug de ingesta, reentrenar aprende el bug; si es estacional, un modelo con features de calendario supera al reentrenamiento reactivo.
4. **«PSI > 0.25 en alguna feature = reentrenar ya.»** El umbral es convención, depende del binning y de la importancia de la feature en el modelo; una feature irrelevante puede derivar sin efecto alguno.
5. **«Las etiquetas de producción llegan limpias.»** Llegan tarde y sesgadas (solo observas el resultado de lo que aprobaste); la evaluación por cohortes y el muestreo aleatorio existen para corregir ese sesgo.

🚀 Del aprendizaje a la operación

El laboratorio calcula deriva sobre distribuciones simuladas; en producción se añaden ventanas deslizantes con estacionalidad, cientos de features monitoreadas (control de falsas alarmas por comparaciones múltiples), herramientas como Evidently o los monitores de las plataformas cloud, y el bucle organizativo: quién investiga una alerta de PSI, con qué presupuesto de etiquetado continuo y qué autoridad para disparar el reentrenamiento. El costo real del sistema no es calcular PSI: es mantener el bucle de etiquetas vivo.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("evaluation")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Gama et al. (2014), "A Survey on Concept Drift Adaptation", ACM Computing Surveys — uso: fuente primaria del mecanismo estudiado
- Evidently AI Documentation — monitoreo de deriva — uso: referencia consultada en su fuente original
- Huyen, *Designing Machine Learning Systems* — cap. de distribution shifts y monitoreo — uso: referencia consultada en su fuente original
- Breck et al. (2017), "The ML Test Score", IEEE Big Data — tests de monitoreo — uso: referencia consultada en su fuente original
- Google, "Rules of Machine Learning" — reglas de monitoreo (parte III) — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P110 · Una revisión sobre adaptación a la deriva de concepto	2014	Ordena el problema de que el mundo cambie después de entrenar, y separa detectar de adaptarse.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Huyen, Chip — *Designing Machine Learning Systems*	2022	ISBN 9781098107956 · web de la obra · *pendiente de confirmar en su catálogo*	citada en las referencias de esta clase · obra de referencia de la parte 12

📝 Evaluación completa

❓ Preguntas

1. Define deriva, feedback y evaluación continua sin usar una marca o framework como definición.
2. Explica la relación entre drift, feedback, continuous eval, monitoring.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;

- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

155 — LLMOps y gestión de prompts

Parte: 12 — Ingeniería de IA, MLOps, LLMOps y AgentOps
Nivel: experto · **Horas estimadas:** 6
Laboratorio: `observability` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **llmops y gestión de prompts** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar llmops y gestión de prompts usando los conceptos `LLMOps`, `prompts`, `versions`, `evals`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`LLMOps`, `prompts`, `versions`, `evals`

🗺️ Ubicación en el mapa de la IA

En un sistema con LLM, el artefacto que más cambia no son los pesos (que suelen ser de un proveedor) sino el **prompt** y su configuración. LLMOps traslada la disciplina de MLOps (145-151) a ese nuevo artefacto: versionarlo, evaluarlo por regresión antes de cada cambio y observarlo en producción. Esta clase toma la evaluación de LLMs (parte 06) y la convierte en proceso operado, preparando el terreno de AgentOps (156).

📖 Fundamentos

📦 El prompt es un artefacto desplegable

Lo que se versiona no es solo el texto: es la **configuración completa de la llamada**, porque cualquiera de sus campos cambia el comportamiento:

```
prompt_config: soporte-respuestas
version: v12          # inmutable una vez publicada
model: claude-sonnet-4-5      # el modelo es parte del artefacto
params: {max_tokens: 1024, temperature: 0.2}
system: |
  Eres un agente de soporte de ACME. Responde solo con información
  del contexto. Si no está en el contexto, dilo explícitamente.
template: "Contexto:\n{contexto}\n\nPregunta: {pregunta}"
changelog: "v12: añade instrucción anti-alucinación; v11 inventaba políticas"
```

Principios heredados del registro de modelos (150): versiones **inmutables** (v12 no se edita: se crea v13), despliegue por referencia (producción apunta a una versión, no a un texto pegado en el código), rollback = re-apuntar, y linaje: cada respuesta en producción registra `prompt_version` en su span (clase 153).

🧪 Evals de regresión

Editar un prompt es editar código sin compilador: la única red es una **suite de evals** que corre antes de publicar, como los tests en CI (151):

1. **Dataset de evaluación curado**: casos reales + casos límite + casos adversarios, con salida esperada o criterios de aceptación. Crece con cada incidente (cada bug reportado se convierte en caso).
2. **Calificadores (graders)**: exactos (¿el JSON parsea?, ¿contiene la cláusula obligatoria?), programáticos (similitud, regex), o **LLM-as-judge** con rúbrica — este último se calibra periódicamente contra juicios humanos y se fija su versión de modelo (un juez que cambia solo introduce ruido en la serie histórica).
3. **Comparación A/B de versiones**: `eval(v13) vs eval(v12)` sobre el mismo dataset; se publica solo si no hay regresión en los criterios bloqueantes.

```
publicar v13 si:
  aprobados(v13) ≥ aprobados(v12) en criterios bloqueantes (formato, seguridad)
  y score_medio(v13) ≥ score_medio(v12) − tolerancia en criterios blandos
  y costo y latencia dentro de presupuesto (tokens de entrada del prompt nuevo)
```

🎲 El problema del no determinismo

Con `temperature > 0` la misma entrada produce salidas distintas: una eval seria corre cada caso k veces (o a temperature 0 cuando el caso lo permite) y reporta tasa de aprobación, no un booleano. La regresión se declara sobre la tasa: «v13 aprueba el caso de reembolsos 9/10 veces; v12 lo aprobaba 10/10» es una señal, una corrida única no.

🔁 Deriva sin tocar nada

Particularidad de LLMOps: el comportamiento puede cambiar **sin que tú cambies nada** — el proveedor actualiza el modelo, o deprecia la versión anclada. Defensas: fijar la versión exacta del modelo en la configuración, re-correr la suite ante cualquier anuncio del proveedor, y monitorear en producción proxies de calidad (tasa de respuestas «no está en el contexto», longitud media, tasa de fallos de formato) que delatan un cambio de comportamiento aguas arriba.

🧮 Ejemplo trabajado

El equipo edita el prompt de soporte para reducir alucinaciones. Suite: 60 casos (40 normales, 12 límite, 8 adversarios), k = 5 corridas por caso, juez con rúbrica fijada + 2 graders exactos (formato JSON, presencia de descargo legal).

```
                         v12               v13 (candidata)
formato JSON válido      300/300           300/300        bloqueante ✓
descargo legal presente  298/300           300/300        bloqueante ✓
fidelidad al contexto    0.86 (juez)       0.93 (juez)    objetivo ↑ ✓
utilidad percibida       0.81 (juez)       0.74 (juez)    blando: −0.07 ✗ tolerancia 0.05
tokens de entrada        310               415            +34 % de costo por llamada
```

Decisión: **no publicar todavía**. La mejora en fidelidad (el objetivo del cambio) es real, pero la utilidad cayó más que la tolerancia: el prompt nuevo hace al asistente tan conservador que responde «no está en el contexto» en casos donde sí estaba (se verifica leyendo las transcripciones de los 6 casos que empeoraron — las evals dan números, las transcripciones dan el porqué). Iteración: v14 relaja la instrucción («si la respuesta está parcialmente, respóndela y señala el límite»), recupera utilidad 0.80 con fidelidad 0.92 y se publica. El costo +34 % se acepta y queda registrado.

📊 Propiedades y comparación

Aspecto	MLOps (modelo propio)	LLMOps (LLM de proveedor)
Artefacto que más cambia	pesos + features	prompt + configuración + versión de modelo
Ciclo de cambio	días-semanas (entrenar)	minutos (editar texto) — por eso más peligroso
Test previo al despliegue	eval sobre dataset etiquetado	suite de evals con graders + juez
Determinismo	alto (seed propia)	bajo: k corridas por caso, tasas no booleanos
Deriva exógena	no (los pesos son tuyos)	sí: el proveedor puede cambiar el modelo
Rollback	re-desplegar versión anterior	re-apuntar a prompt_version anterior (segundos)
Costo marginal	infraestructura propia	por token: el tamaño del prompt es costo directo

⚠️ Errores conceptuales frecuentes

1. **«Probé el prompt nuevo con tres ejemplos en el playground y mejora.»** Sin suite ni k corridas, es anécdota: los prompts mejoran unos casos y rompen otros en silencio (por eso el ejemplo trabajado existe).

2. **«El prompt vive en el código.»** Hardcodearlo acopla su ciclo de cambio al del despliegue de software e impide rollback independiente y linaje por versión.
3. **«El juez LLM es un oráculo.»** Es un modelo con sesgos (longitud, posición, autopreferencia); se calibra contra humanos, se fija su versión y se auditan sus discrepancias.
4. **«Temperature 0 me da evals deterministas, luego una corrida basta.»** Reduce la varianza pero no la elimina (empates numéricos, no determinismo de inferencia), y evalúa un régimen que quizá no es el de producción.
5. **«Si no cambio nada, nada cambia.»** El modelo del proveedor puede cambiar bajo tus pies; sin versión fijada y suite re-ejecutable, ni lo detectarás.

🚀 Del aprendizaje a la operación

El laboratorio simula el ciclo editar → evaluar → publicar con datos deterministas; en producción se añade un gestor de prompts con control de acceso (quién puede publicar a producción), evals en CI con presupuesto de tokens real, calibración periódica del juez contra muestras humanas, y la cultura que más cuesta: convertir cada incidente de producción en un caso permanente de la suite, para que el mismo bug no regrese dos veces.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("observability")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Anthropic — Documentación: prompt engineering y evaluaciones — uso: referencia consultada en su fuente original
- Zheng et al. (2023), "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena" (arXiv:2306.05685) — uso: fuente primaria del mecanismo estudiado
- OpenTelemetry — Semantic Conventions for Generative AI — uso: marco normativo de referencia
- MLflow Documentation — tracking y evaluación de LLMs — uso: referencia consultada en su fuente original
- Ribeiro et al. (2020), "Beyond Accuracy: Behavioral Testing of NLP Models with CheckList" (arXiv:2005.04118) — uso: fuente primaria del mecanismo estudiado

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P116 · Por qué Johnny no sabe hacer prompts: cómo los no expertos intentan (y fallan) diseñar prompts	2023	Documenta con usuarios reales que iterar prompts sin conjunto de evaluación produce mejoras imaginarias, y por qué la intuición falla sistemáticamente.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Huyen, Chip — *Designing Machine Learning Systems*	2022	ISBN 9781098107956 · web de la obra · *pendiente de confirmar en su catálogo*	obra de referencia de la parte 12 · toda la parte

Normas y documentación oficial que aplica esta clase: OpenTelemetry

📝 Evaluación completa

❓ Preguntas

1. Define llmops y gestión de prompts sin usar una marca o framework como definición.
2. Explica la relación entre LLMOps, prompts, versions, evals.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

156 — AgentOps y análisis de trayectorias

Parte: 12 — Ingeniería de IA, MLOps, LLMOps y AgentOps
Nivel: experto · **Horas estimadas:** 6
Laboratorio: `observability` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **agentops y análisis de trayectorias** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar agentops y análisis de trayectorias usando los conceptos `AgentOps`, `trajectories`, `tools`, `traces`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`AgentOps`, `trajectories`, `tools`, `traces`

🗺️ Ubicación en el mapa de la IA

Un LLM responde una llamada; un **agente** ejecuta una trayectoria: decide, usa herramientas, observa y vuelve a decidir (parte 09). AgentOps extiende LLMOps (155) a ese objeto más difícil: ya no se evalúa una salida, sino un **camino completo** con longitud variable, costo variable y efectos en el mundo. Sus trazas son las de la clase 153 con semántica extra, y sus métricas son las que el proyecto integrador (159) tendrá que exponer en un panel.

📖 Fundamentos

🏞️ La trayectoria como unidad de análisis

Una **trayectoria** es la secuencia registrada de una tarea:

```
tarea T-4471: "reembolsa el pedido 9912 si procede"
  paso 1  razonamiento → tool: consultar_pedido(9912)        → ok (320 ms)
  paso 2  razonamiento → tool: politica_reembolsos(cat=ropa) → ok (180 ms)
  paso 3  razonamiento → tool: crear_reembolso(9912, 100%)   → error: monto>límite
  paso 4  razonamiento → tool: crear_reembolso(9912, 80%)    → ok
  paso 5  respuesta final al usuario                          éxito, 4 pasos
  totales: 5 llamadas LLM, 11 200 tokens in / 840 out, 9.4 s, $0.041
```

Se instrumenta como una traza OTel: un span raíz por tarea, spans hijos por llamada LLM (`gen_ai.*`) y por herramienta (nombre, argumentos, resultado, error). Sin esta estructura solo sabrías que la tarea «tardó 9 s»; con ella puedes preguntar *dónde* se gastan los pasos y *qué* herramienta falla.

📊 Métricas de agente

Las cuatro familias que definen la salud de un agente, siempre como distribuciones:

1. **Éxito de tarea**: tasa de tareas completadas correctamente (definida por eval, no por «el agente dijo que terminó» — los agentes declaran éxito con optimismo).
2. **Pasos por tarea**: distribución (p50/p95). Colas largas delatan bucles, herramientas confusas o tareas fuera de alcance.
3. **Costo por tarea**: tokens y $ (suma de todas las llamadas). Un agente puede tener 85 % de éxito y ser inviable a $0.80/tarea.
4. **Intervenciones**: cuántas veces un humano corrigió, aprobó o abortó — la métrica que mide autonomía *real* y que decide dónde relajar o endurecer los límites.

Métricas de diagnóstico derivadas: tasa de error por herramienta, tasa de recuperación tras error (¿el paso 3 fallido se corrige en el 4, como arriba, o se repite?), tokens «desperdiciados» en pasos que no aportaron al resultado.

🔬 Análisis de trayectorias

El análisis agregado responde *cuánto*; el de trayectorias responde *por qué*:

- **Clustering de fallos**: agrupar trayectorias fallidas por patrón (misma herramienta, mismo tipo de error, mismo punto de abandono). Tres incidentes distintos suelen ser un solo bug de descripción de herramienta.
- **Puntos de divergencia**: comparar trayectorias exitosas y fallidas de la misma familia de tareas y localizar el primer paso donde difieren.
- **Replay**: re-ejecutar la misma tarea contra una configuración nueva (prompt de sistema, set de herramientas) — el champion-challenger (150) aplicado a agentes, con la cautela de que las herramientas con efectos reales se *mockean* en el replay.

🚨 Evaluación continua de agentes

Igual que en 152 pero sobre tareas: suite de tareas de referencia con criterios de éxito verificables, ejecutada ante cada cambio de prompt, herramienta o modelo base. La guía de Anthropic sobre agentes efectivos insiste en el prerequisito: empezar simple e **instrumentar desde el primer día** — un agente sin trazas es indepurable por diseño.

🌙 Observabilidad de agentes siempre activos

Un agente que trabaja sin una sesión humana abierta necesita métricas adicionales: horizonte sin supervisión, wakeups intentados/exitosos, edad del objetivo, rechazos por autorización vencida, latencia de cancelación, gasto por periodo, efectos externos por ejecución y tiempo hasta que una persona reconoce una escalación. La unidad de análisis pasa de una trayectoria aislada a la cadena `evento -> reanudación -> decisión -> efecto -> notificación`.

Las ejecuciones programadas deben compartir un correlation ID estable y producir un checkpoint auditable, pero no conservar secretos o prompts completos solo para facilitar debugging.

🧮 Ejemplo trabajado

Agente de soporte, semana de 1 000 tareas:

```
métrica                  valor        lectura
éxito (eval)             84.0 %       ¿bien? depende del costo del 16 % restante
pasos p50 / p95          4 / 19       p95 ≈ 5× p50 → hay bucles
costo medio / p95        $0.05 / $0.31
intervenciones           9.0 %        1 de cada 11 tareas necesitó un humano
```

Clustering de las 160 tareas fallidas:

```
cluster A  62 fallos  crear_reembolso devuelve "monto>límite" y el agente reintenta
                      el MISMO monto hasta agotar presupuesto de pasos (bucle)
cluster B  41 fallos  la tarea exige consultar envíos: herramienta inexistente
cluster C  57 fallos  variados (larga cola)
```

Acciones quirúrgicas, no «mejorar el prompt» en abstracto: para A, incluir el límite máximo en el mensaje de error de la herramienta (el agente reintentaba a ciegas porque el error no decía el límite) — fallos A caen a 8; para B, o añadir la herramienta de envíos o declarar la tarea fuera de alcance y derivarla a humano en el paso 1 (mejor 9 % de intervención temprana que 19 pasos de agonía). El p95 de pasos baja de 19 a 9. Nada de esto era visible en la tasa de éxito agregada: salió de leer trayectorias.

📊 Propiedades y comparación

Aspecto	LLMOps (llamada única)	AgentOps (trayectoria)
Unidad evaluada	respuesta	tarea completa (multi-paso)
Métrica primaria	calidad de la salida	éxito de tarea verificado
Costo	por llamada, acotado	por tarea, variable (pasos × llamadas)
No determinismo	una muestra por salida	se compone: bifurca en cada paso
Fallo típico	respuesta incorrecta	bucles, herramienta equivocada, éxito declarado falso
Depuración	leer la respuesta	leer la trayectoria (spans LLM + tools)
Seguridad	contenido	contenido + **acciones con efectos** (límites, aprobaciones)

⚠️ Errores conceptuales frecuentes

1. **«El agente dijo que completó la tarea, cuento éxito.»** El éxito auto-reportado sobreestima sistemáticamente; el éxito se verifica con un criterio externo (estado del sistema, eval, humano muestreado).
2. **«Optimizar la tasa de éxito agregada.»** Sin clustering de fallos, se «mejora el prompt» a ciegas; los fallos de agentes vienen en familias con causas concretas (herramienta opaca, capacidad ausente), como muestra el ejemplo.
3. **«Menos pasos siempre es mejor.»** Un agente puede acortar trayectorias saltándose verificaciones; pasos p50 se lee junto con éxito e intervenciones, nunca solo.
4. **«Las intervenciones humanas son ruido a eliminar.»** Son la señal de autonomía real y la red de seguridad: la meta es reducirlas *donde el agente demuestra fiabilidad*, no suprimir el mecanismo.
5. **«Replay de trayectorias = repetir la tarea.»** Sin mockear herramientas con efectos (pagos, correos), el replay re-ejecuta efectos reales; y con estado del mundo cambiado, la comparación puede no ser válida.

🚀 Del aprendizaje a la operación

El laboratorio analiza trayectorias sintéticas deterministas; producción añade plataformas de trazas para agentes (backends OTel, LangSmith y equivalentes), muestreo de trayectorias largas, redacción de PII en argumentos de herramientas, presupuestos duros por tarea (pasos, tokens, $) aplicados en el runtime, y revisión humana periódica de trayectorias muestreadas — porque las métricas dicen cuánto falla el agente, pero solo leer trayectorias enseña *cómo* falla.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("observability")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;

- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Anthropic, "Building Effective Agents" — uso: referencia consultada en su fuente original
- Anthropic — Documentación: agentes y uso de herramientas — uso: referencia consultada en su fuente original
- OpenTelemetry — Semantic Conventions for Generative AI — uso: marco normativo de referencia
- Yao et al. (2023), "ReAct: Synergizing Reasoning and Acting in Language Models" (arXiv:2210.03629) — uso: fuente primaria del mecanismo estudiado
- Model Context Protocol — especificación — uso: marco normativo de referencia

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P117 · AgentBench: evaluar modelos de lenguaje como agentes	2023	Evalúa agentes en ocho entornos distintos y hace visible que la tasa agregada esconde dónde y cómo fallan.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Huyen, Chip — *Designing Machine Learning Systems*	2022	ISBN 9781098107956 · web de la obra · *pendiente de confirmar en su catálogo*	obra de referencia de la parte 12 · toda la parte

Normas y documentación oficial que aplica esta clase: OpenTelemetry · Model Context Protocol

📝 Evaluación completa

❓ Preguntas

1. Define agentops y análisis de trayectorias sin usar una marca o framework como definición.
2. Explica la relación entre AgentOps, trajectories, tools, traces.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

157 — Costo, latencia, caching y capacidad

Parte: 12 — Ingeniería de IA, MLOps, LLMOps y AgentOps
Nivel: experto · **Horas estimadas:** 6
Laboratorio: `observability` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **costo, latencia, caching y capacidad** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar costo, latencia, caching y capacidad usando los conceptos `cost`, `latency`, `cache`, `capacity`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`cost`, `latency`, `cache`, `capacity`

🗺️ Ubicación en el mapa de la IA

Después de saber desplegar (152), observar (153) y evaluar continuamente (151-153) un sistema de IA, esta clase aborda la restricción que decide si el sistema sobrevive: la **economía de la inferencia**. Los LLM cobraron por token lo que antes era un costo fijo de servir un modelo propio, y convirtieron el costo y la latencia en variables de diseño de producto. Lo que se aprende aquí — modelo de costo por token, percentiles de latencia, caching y planificación de capacidad — alimenta directamente la resiliencia (158) y el proyecto final de plataforma observable (159).

📖 Fundamentos

💰 Modelo de costo por token

En un LLM servido por API el costo de una petición es lineal en tokens, con precios distintos para entrada y salida (la salida es más cara porque se genera token a token):

```
costo_peticion = t_in · p_in + t_out · p_out
costo_mensual  = N · costo_peticion_promedio

t_in, t_out : tokens de entrada / salida
p_in, p_out : precio por token (usualmente cotizado por millón de tokens, MTok)
N           : peticiones por mes
```

Tres consecuencias de la linealidad: (1) el prompt de sistema se paga en **cada petición**, por lo que recortarlo o cachearlo tiene efecto multiplicativo; (2) limitar `max_tokens` acota el peor caso de costo y de latencia; (3) el costo por petición varía con la longitud, así que se presupuesta con la distribución real de tráfico, no con un promedio inventado.

⏱️ Latencia: percentiles, no promedios

La latencia de inferencia se descompone en **TTFT** (time to first token, dominado por el procesamiento del prompt) y **tiempo por token de salida** (generación autoregresiva). Se reporta con percentiles:

- **p50** (mediana): experiencia típica.
- **p95 / p99**: experiencia de cola; con muchas peticiones por sesión, casi todo usuario toca la cola (si una página hace 10 llamadas, la probabilidad de que al menos una caiga sobre el p95 es $1 - 0.95^{10} \approx 40\ \%$).

El promedio es engañoso porque la distribución tiene cola pesada: un p50 de 800 ms con p99 de 12 s da un promedio «aceptable» y usuarios furiosos. Los SLO (Google SRE) se definen sobre percentiles: «p95 < 2 s durante 30 días».

🗂️ Caching en sistemas de IA

Tipo	Qué guarda	Acierta cuando	Riesgo
Caché exacta	respuesta por hash del prompt	prompt idéntico byte a byte	invalidación al cambiar modelo/prompt
Prompt caching (API)	prefijo procesado (KV cache)	prefijo compartido (sistema + contexto)	requiere prefijos estables
Caché semántica	respuesta por similitud de embedding	paráfrasis de la misma pregunta	servir respuesta incorrecta a pregunta parecida

El ahorro esperado con tasa de aciertos `h` y costo de acierto casi nulo: `costo_efectivo ≈ (1 − h) · costo_peticion + h · costo_lookup`. Una caché semántica exige umbral de similitud calibrado y métrica de *falsos aciertos*, porque un acierto incorrecto es un error silencioso de calidad.

📈 Planificación de capacidad

Con llegadas de λ peticiones/s y tiempo medio de servicio S, la ley de Little da la concurrencia media `L = λ · W` (W = tiempo en el sistema). La utilización `ρ = λ · S / c` (c = servidores o slots de inferencia) debe mantenerse lejos de 1: en colas, la espera crece de forma no lineal al acercarse a saturación. La práctica: dimensionar para el pico previsto con margen (p. ej. $\rho \le 0.6$-0.7 en pico), definir límites de tasa por cliente y una política de degradación (modelo más pequeño, cola, rechazo explícito) antes de saturar.

🧮 Ejemplo trabajado

Asistente interno: 200 000 peticiones/mes; promedio 1 800 tokens de entrada (1 200 de ellos son un prefijo estable de sistema + contexto) y 350 de salida. Precios ilustrativos: p_in = 3 USD/MTok, p_out = 15 USD/MTok; el prefijo cacheado se cobra a 0.3 USD/MTok con tasa de acierto 0.9.

```
Sin caché:
  costo_peticion = 1800·(3/1e6) + 350·(15/1e6)
                 = 0.0054 + 0.00525 = 0.01065 USD
  costo_mensual  = 200000 · 0.01065 = 2 130 USD

Con prompt caching (0.9 de aciertos sobre los 1 200 tokens de prefijo):
  entrada = 600·(3/1e6)                      (tokens no cacheables)
          + 0.9·1200·(0.3/1e6)               (prefijo con acierto)
          + 0.1·1200·(3/1e6)                 (prefijo sin acierto)
          = 0.0018 + 0.000324 + 0.00036 = 0.002484 USD
  costo_peticion = 0.002484 + 0.00525 = 0.007734 USD
  costo_mensual  = 200000 · 0.007734 ≈ 1 547 USD   (−27 %)
```

Capacidad: pico de $\lambda = 5$ peticiones/s con $S = 2$ s por petición y 4 peticiones concurrentes por réplica → concurrencia media $L = \lambda \cdot S = 10$; réplicas mínimas = 10/4 = 2.5 → 3 réplicas a $\rho \approx 0.83$: demasiado justo; con margen ($\rho \leq 0.6$) se dimensionan 5 réplicas. La decisión queda documentada con sus supuestos.

📊 Propiedades y comparación

Palanca	Reduce costo	Reduce latencia	Riesgo de calidad	Esfuerzo
Recortar prompt / max_tokens	Sí	Sí	Medio (pérdida de contexto)	Bajo
Prompt caching	Sí (entrada)	Sí (TTFT)	Nulo	Bajo
Caché semántica	Sí (peticiones enteras)	Sí	Alto (falsos aciertos)	Medio
Modelo más pequeño / enrutamiento	Sí	Sí	Medio-alto (medir con evals)	Medio
Batching / streaming	No directo	Percibida (streaming)	Nulo	Medio
Más réplicas	No (sube)	Sí (colas)	Nulo	Bajo

⚠️ Errores conceptuales frecuentes

1. **«La latencia promedio es buena, el sistema está bien.»** Los usuarios viven en los percentiles; con varias llamadas por sesión, la cola p95/p99 domina la experiencia.
2. **«La caché semántica es ahorro gratis.»** Un acierto con umbral mal calibrado sirve una respuesta a otra pregunta: es un error de calidad silencioso que ninguna métrica de costo detecta.
3. **«Presupuesto con el prompt de prueba.»** El costo real depende de la distribución de longitudes en producción; el prompt de desarrollo suele ser más corto que el contexto real con RAG e historial.
4. **«Dimensionar al 100 % de utilización.»** Cerca de saturación la espera en cola crece de forma no lineal; sin margen, cualquier pico degrada a todos.
5. **«Optimizar costo es solo cambiar de modelo.»** Sin evals (clases 154-155) un modelo más barato puede costar más en correcciones, reintentos y fuga de usuarios que lo que ahorra por token.

🚀 Del aprendizaje a la operación

El laboratorio calcula costos y percentiles sobre tráfico sintético; operar de verdad exige medir tokens y latencia reales por petición (OpenTelemetry, clase 150), atribuir costo por equipo/función, alertar sobre presupuesto y sobre SLO de p95, recalibrar la caché semántica con evals periódicas y revisar precios y límites de tasa del proveedor, que cambian con cada generación de modelos.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("observability")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Anthropic — *Prompt caching* (documentación oficial): https://docs.anthropic.com/en/docs/build-with-claude/prompt-caching
- Google — *Site Reliability Engineering* (Beyer et al., 2016), cap. «Service Level Objectives», libro gratuito: https://sre.google/sre-book/service-level-objectives/
- Dean y Barroso (2013) — *The Tail at Scale*, CACM 56(2): https://dl.acm.org/doi/10.1145/2408776.2408794
- Little (1961) — *A Proof for the Queuing Formula L = λW*, Operations Research 9(3): https://doi.org/10.1287/opre.9.3.383
- Huyen — *Designing Machine Learning Systems* (O'Reilly, 2022): https://www.oreilly.com/library/view/designing-machine-learning/9781098107956/

- OpenTelemetry — documentación oficial (métricas y trazas para medir latencia y uso): https://opentelemetry.io/docs/

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P45 · Destilar el conocimiento de una red neuronal	2015	Las probabilidades del maestro contienen más información que la etiqueta correcta: el modelo pequeño aprende de esa estructura.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Huyen, Chip — *Designing Machine Learning Systems*	2022	ISBN 9781098107956 · web de la obra · *pendiente de confirmar en su catálogo*	citada en las referencias de esta clase · obra de referencia de la parte 12

Normas y documentación oficial que aplica esta clase: OpenTelemetry Documentation

📝 Evaluación completa

❓ Preguntas

1. Define costo, latencia, caching y capacidad sin usar una marca o framework como definición.
2. Explica la relación entre cost, latency, cache, capacity.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

158 — Resiliencia, idempotencia, rollback y recuperación

Parte: 12 — Ingeniería de IA, MLOps, LLMOps y AgentOps
Nivel: experto · **Horas estimadas:** 6
Laboratorio: `workflow` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **resiliencia, idempotencia, rollback y recuperación** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar resiliencia, idempotencia, rollback y recuperación usando los conceptos `resilience`, `idempotency`, `rollback`, `recovery`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`resilience`, `idempotency`, `rollback`, `recovery`

🗺️ Ubicación en el mapa de la IA

Los sistemas de IA en producción dependen de servicios remotos (APIs de modelos, vector stores, colas) que **fallan con normalidad estadística**: la pregunta no es si fallan sino cuándo. Esta clase importa a la ingeniería de IA los patrones de estabilidad que Michael Nygard sistematizó en *Release It!* tras años de sistemas caídos en producción — reintentos con backoff, circuit breaker, bulkheads — y les suma idempotencia, rollback y sagas. Completa la economía de la clase 157 (un reintento mal diseñado duplica costo y latencia) y prepara el proyecto integrador 156.

📖 Fundamentos

🔁 Reintentos con backoff exponencial y jitter

Ante un fallo transitorio (timeout, HTTP 429/503) se reintenta esperando cada vez más, con aleatoriedad para no sincronizar a todos los clientes:

```
espera_n = random(0, min(tope, base · 2^n))      # "full jitter" (AWS)

n     : número de intento (0, 1, 2, …)
base  : espera inicial (p. ej. 0.5 s)
tope  : espera máxima (p. ej. 30 s)
```

Sin jitter, mil clientes que fallaron juntos reintentan juntos y producen una **estampida sincronizada** (thundering herd) que vuelve a tumbar al servicio. Reglas duras: reintentar solo errores transitorios (nunca un 400 o un error de validación), acotar el número de intentos, y presupuestar el peor caso: con 3 intentos y espera media `tope/2`, la latencia p99 puede multiplicarse.

⚡ Circuit breaker

Un reintento protege una petición; el circuit breaker protege al **sistema**. Es una máquina de estados alrededor de la dependencia:

- **Cerrado**: las llamadas pasan; se cuentan los fallos recientes.
- **Abierto**: tras superar el umbral de fallos, las llamadas se rechazan de inmediato (fail fast) sin tocar la dependencia, dándole tiempo a recuperarse.
- **Semiabierto**: pasado un tiempo, se deja pasar una llamada de prueba; si funciona se cierra, si falla se reabre.

Sin breaker, los reintentos de todos los clientes convierten una degradación parcial en una **falla en cascada**: cada capa agota sus hilos esperando a la capa caída (el antipatrón que Nygard llama *integration point* sin defensas).

🔒 Idempotencia

Una operación es idempotente si ejecutarla N veces produce el mismo estado que ejecutarla una vez: `f(f(x)) = f(x)`. Es el **prerrequisito de los reintentos**: reintentar un cobro no idempotente cobra dos veces. Técnica estándar: el cliente genera una *idempotency key* única por operación lógica; el servidor registra la clave y, si se repite, devuelve el resultado original sin re-ejecutar. En agentes: reejecutar un paso de una trayectoria (enviar correo, escribir fila) exige la misma disciplina.

↩️ Rollback y sagas

- **Rollback de despliegue**: volver a la versión anterior (modelo, prompt, código) ante degradación. Requisitos: artefactos versionados e inmutables (clase 150), despliegue gradual con baseline (canary), y datos compatibles hacia atrás — el rollback de código no deshace una migración de esquema.
- **Saga** (Garcia-Molina y Salem, 1987): una transacción larga se parte en pasos $T_1 \dots T_n$, cada uno con una **compensación** $C_1 \dots C_n$. Si falla T_k, se ejecutan $C_{k-1} \dots C_1$ en orden inverso. No hay aislamiento ACID: la compensación es una acción nueva que revierte el efecto (cancelar reserva, reembolsar), no un «undo» mágico. Es el modelo natural para trayectorias de agentes con efectos externos: cada herramienta con efecto debe declarar su compensación o marcarse como no compensable (y entonces exigir aprobación previa, clase 144).

🧮 Ejemplo trabajado

Cliente llama a una API de LLM con `base = 0.5 s`, `tope = 30 s`, máximo 4 intentos, full jitter. Secuencia de esperas máximas:

```
intento 0 falla → espera ~ U(0, min(30, 0.5·1))  = U(0, 0.5 s)
intento 1 falla → espera ~ U(0, min(30, 0.5·2))  = U(0, 1 s)
intento 2 falla → espera ~ U(0, min(30, 0.5·4))  = U(0, 2 s)
intento 3 falla → error definitivo al llamador

Peor caso de espera acumulada = 0.5 + 1 + 2 = 3.5 s (+ 4 timeouts)
Espera media acumulada ≈ 0.25 + 0.5 + 1 = 1.75 s
```

Circuit breaker con umbral «5 fallos en 30 s» y ventana de recuperación de 60 s: si la API cae del todo, tras ~5 llamadas el breaker abre y las siguientes fallan en <1 ms hacia el fallback (respuesta en caché o modelo local), en lugar de gastar 3.5 s + timeouts por petición. La saga del agente «reservar viaje»: T_1 reservar vuelo / C_1 cancelar vuelo; T_2 reservar hotel / C_2 cancelar hotel; T_3 cobrar / C_3 reembolsar. Si T_3 falla definitivamente, se ejecutan C_2 y C_1; la idempotency key por paso evita duplicados si la compensación misma se reintenta.

📊 Propiedades y comparación

Patrón	Protege contra	Ámbito	Requiere	Riesgo si se usa mal
Retry + backoff + jitter	Fallos transitorios	Una petición	Operación idempotente	Amplificar carga y costo
Circuit breaker	Falla en cascada	Una dependencia	Umbrales calibrados	Abrir por fallos no representativos
Idempotency key	Efectos duplicados	Una operación con efecto	Almacén de claves	Clave mal elegida = deduplicar de más
Rollback	Versión degradada	Despliegue	Artefactos versionados, canary	Esquemas de datos incompatibles
Saga	Transacción larga fallida	Flujo multi-paso	Compensación por paso	Pasos sin compensación posible

⚠️ Errores conceptuales frecuentes

1. **«Reintentar siempre es inofensivo.»** Sin idempotencia duplica efectos; sin jitter sincroniza clientes; sin límite convierte una degradación en ataque de denegación autoinfligido.
2. **«El circuit breaker es un retry sofisticado.»** Son complementarios y de ámbito distinto: el retry insiste por una petición; el breaker deja de insistir por todo el sistema para proteger a la dependencia.

3. **«Idempotencia = la operación no hace nada dos veces.»** Hace lo mismo: el estado final es idéntico y el cliente recibe el mismo resultado; no es lo mismo que ignorar la segunda llamada con un error.
4. **«El rollback deshace todo.»** Revierte el artefacto desplegado, no los efectos ya producidos (filas escritas, correos enviados, migraciones); para efectos se necesitan compensaciones (saga).
5. **«Las sagas dan garantías ACID.»** No hay aislamiento: estados intermedios son visibles y la compensación puede fallar; se diseña para *consistencia eventual* con reintentos idempotentes de la compensación.

🚀 Del aprendizaje a la operación

El laboratorio simula fallos con semilla fija; en producción los patrones se implementan con bibliotecas probadas (resiliencia del SDK del proveedor, tenacity, Polly, mallas de servicio), los umbrales del breaker se calibran con telemetría real (clase 153), cada reintento se registra con su costo (clase 154), y los runbooks de rollback se ensayan — un rollback que nunca se probó es una hipótesis, no un plan de recuperación.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("workflow")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Nygard — *Release It! Design and Deploy Production-Ready Software* (2.ª ed., Pragmatic Bookshelf, 2018): patrones de estabilidad (circuit breaker, bulkhead, timeout). https://pragprog.com/titles/mnee2/release-it-second-edition/
- Garcia-Molina y Salem (1987) — *Sagas*, SIGMOD '87: https://doi.org/10.1145/38713.38742
- AWS Architecture Blog — *Exponential Backoff and Jitter* (análisis comparativo de estrategias de jitter): https://aws.amazon.com/blogs/architecture/exponential-backoff-and-jitter/
- Google — *Site Reliability Engineering*, cap. «Addressing Cascading Failures», libro gratuito: https://sre.google/sre-book/addressing-cascading-failures/
- Anthropic — manejo de errores y límites de tasa de la API (códigos de error y cabeceras de reintento): https://docs.anthropic.com/en/api/errors

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P108 · CAP doce años después: cómo han cambiado las «reglas»	2012	Corrige la lectura simplista de su propio teorema: no se eligen dos de tres, se elige por operación y solo mientras dura la partición.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Huyen, Chip — *Designing Machine Learning Systems*	2022	ISBN 9781098107956 · web de la obra · *pendiente de confirmar en su catálogo*	obra de referencia de la parte 12 · toda la parte

📝 Evaluación completa

❓ Preguntas

1. Define resiliencia, idempotencia, rollback y recuperación sin usar una marca o framework como definición.
2. Explica la relación entre resilience, idempotency, rollback, recovery.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;

- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

159 — Proyecto: plataforma de IA observable

Parte: 12 — Ingeniería de IA, MLOps, LLMOps y AgentOps
Nivel: experto · **Horas estimadas:** 10
Laboratorio: `capstone` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **proyecto: plataforma de ia observable** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar proyecto: plataforma de ia observable usando los conceptos `platform`, `observability`, `release`, `SLO`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`platform`, `observability`, `release`, `SLO`

🗺️ Ubicación en el mapa de la IA

Este proyecto cierra la parte 12 integrando las once clases anteriores en un solo artefacto: una **plataforma de IA observable**, el sustrato que separa a las organizaciones que operan IA de las que solo la demuestran. La lección de fondo viene de Sculley et al. (2015): en un sistema real de ML el modelo es una fracción pequeña del código; lo que domina es la infraestructura de datos, serving, monitoreo y operación. La parte 13 construirá encima la evaluación, la seguridad y la gobernanza — que solo son posibles si esta capa produce evidencia trazable.

📖 Fundamentos

🧱 Qué es una plataforma de IA observable

Una plataforma es el conjunto de capacidades compartidas que todo caso de uso de IA de la organización reutiliza en lugar de reinventar. El proyecto integra las piezas de las clases 148-158 como un pipeline único:

1. **Ciclo de vida y trazabilidad (145-146)**: cada artefacto (dataset, modelo, prompt, configuración de agente) tiene versión, linaje y semillas registradas. Regla: si no puedes decir *qué* produjo una

predicción, no puedes depurarla ni auditarla.
2. **Registro y promoción (150)**: los artefactos pasan por etapas (challenger → champion) con criterios de promoción explícitos y comparación contra baseline, no por antigüedad ni entusiasmo.
3. **CI/CD y pruebas (151)**: cada cambio — código, datos, prompt — dispara pruebas unitarias, de contrato y de comportamiento (evals) antes de llegar a producción.
4. **Serving (152)**: online, batch o streaming según el caso, detrás de un contrato estable que permite cambiar el modelo sin cambiar a los clientes.
5. **Observabilidad (153)**: logs estructurados, métricas y trazas distribuidas (OpenTelemetry) con atributos específicos de IA: versión de modelo y de prompt, tokens, costo, latencia por span, id de trayectoria.
6. **Evaluación continua y deriva (151-153)**: la calidad no se mide una vez sino como serie temporal; deriva de datos y de comportamiento disparan alertas y re-evaluación; las trayectorias de agentes se analizan como trazas.
7. **Economía y resiliencia (154-155)**: presupuesto y SLO por servicio; reintentos, breakers y rollbacks ensayados.

📐 SLO y presupuesto de error

Un **SLI** es una medición (proporción de peticiones bajo 2 s); un **SLO** es el objetivo sobre el SLI (99 % mensual); el **presupuesto de error** es el complemento (1 % ≈ 7.3 h/mes de incumplimiento tolerado). El presupuesto convierte la fiabilidad en una moneda de decisión: si se está gastando demasiado rápido, se congelan lanzamientos y se invierte en estabilidad. En IA se añaden SLO de **calidad** (tasa de aprobación de evals en producción) y de **costo** (USD por 1 000 peticiones), no solo de disponibilidad y latencia.

🚦 Release con evidencia: el ciclo completo

```
cambio (código | datos | prompt | modelo)
  → CI: pruebas + evals offline contra baseline      (148, 151)
  → registro: nueva versión candidata (challenger)    (146, 147)
  → canary: x % de tráfico con telemetría comparada   (149, 150)
  → decisión con datos: promover | rollback           (147, 155)
  → operación: SLO, deriva, costo, trayectorias       (150-154)
```

La propiedad integradora: **cada flecha produce evidencia** (reporte de evals, diff de métricas canary vs. champion, traza del rollback). Una plataforma donde las decisiones de release no dejan evidencia no es observable, por más dashboards que tenga.

🔁 Gate de release para agentes durables

La plataforma debe probar además reinicio, reanudación idempotente, evento duplicado, credencial revocada, política modificada entre checkpoints, presupuesto periódico agotado y cancelación mientras una tool está en vuelo. El release falla si un agente reanudado puede usar permisos antiguos, repetir un efecto irreversible o quedar activo sin owner y sin deadline.

🧮 Ejemplo trabajado

Se quiere promover el prompt `v12` sobre el champion `v11` en un asistente con SLO: p95 < 2 s, aprobación de evals ≥ 90 %, costo ≤ 12 USD/1k peticiones.

Etapa	Evidencia producida	Resultado
Evals offline (n=500)	v12: 93 % vs. v11: 89 %	pasa (Δ+4 pts)
Registro	`prompt:v12` etiquetado challenger, linaje al commit	trazable
Canary 10 %, 48 h	p95: 1.7 s vs. 1.6 s; costo 11.2 vs. 10.8 USD/1k; aprobación online 91 % vs. 88 %	dentro de SLO
Decisión	promoción aprobada; v11 queda como objetivo de rollback	registrada
Día 5	alerta de deriva: aprobación online cae a 84 %	presupuesto de error gastándose
Respuesta	rollback a v11 en minutos (artefacto inmutable, 155) + análisis de trayectorias (156)	causa: nuevo tipo de consulta, no v12

Nota honesta: el rollback no recuperó la calidad (la causa era deriva de datos, no el prompt). La evidencia por etapa es lo que permitió distinguirlo en horas y no en semanas.

📊 Propiedades y comparación

Enfoque	Tiempo hasta detectar degradación	Atribución de causa	Costo de operación	Riesgo principal
Scripts ad hoc por proyecto	Semanas (usuarios se quejan)	Casi imposible	Bajo al inicio, explota después	Deuda técnica oculta (Sculley 2015)
Monitoreo de infraestructura solamente	Horas para caídas, nunca para calidad	Parcial (sin versión de modelo/prompt)	Medio	Degradación de calidad invisible
Plataforma observable (esta clase)	Minutos-horas (SLO + evals online)	Alta (linaje + trazas + versiones)	Alto al inicio, amortizado	Sobre-ingeniería para un solo caso de uso

⚠️ Errores conceptuales frecuentes

1. **«Observabilidad = dashboards.»** Un dashboard muestra lo que alguien decidió graficar; observabilidad es poder responder preguntas nuevas con la telemetría existente (trazas con atributos ricos, no solo paneles).
2. **«El modelo es la plataforma.»** Sculley et al. (2015): el código de ML es una fracción pequeña del sistema; ignorar la infraestructura circundante acumula deuda técnica oculta con intereses compuestos.

3. **«Un SLO del 100 % es el ideal.»** Un objetivo de perfección elimina el presupuesto de error y con él la capacidad de lanzar cambios; la fiabilidad extra por encima de lo que el usuario nota tiene costo y no tiene valor.
4. **«Las evals offline garantizan la calidad online.»** El tráfico real deriva (154); las evals offline son condición necesaria de promoción, no suficiente — por eso existe el canary con evaluación online.
5. **«El rollback resuelve cualquier degradación.»** Solo revierte el artefacto; si la causa es deriva de datos o un fallo de dependencia, la plataforma debe poder distinguirlo con linaje y trazas (el ejemplo trabajado lo muestra).

🚀 Del aprendizaje a la operación

El laboratorio integra los conceptos sobre un pipeline simulado con semilla fija; una plataforma real exige además: multi-tenancy con cuotas y presupuestos por equipo, control de acceso y auditoría sobre el registro de artefactos, retención y privacidad de la telemetría (los prompts pueden contener datos personales), guardias de despliegue integrados con el sistema de incidentes, y un equipo de plataforma que la trate como producto interno con sus propios SLO.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("capstone")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Sculley et al. (2015) — *Hidden Technical Debt in Machine Learning Systems*, NeurIPS 28: https://papers.nips.cc/paper/2015/hash/86df7dcfd896fcaf2674f757a2463eba-Abstract.html
- OpenTelemetry — documentación oficial (trazas, métricas, logs y convenciones semánticas): https://opentelemetry.io/docs/
- Google — *Site Reliability Engineering* (Beyer et al., 2016), caps. «Service Level Objectives» y «Embracing Risk», libro gratuito: https://sre.google/sre-book/table-of-contents/
- MLflow — documentación oficial (tracking, registro de modelos y evaluación): https://mlflow.org/docs/latest/
- Huyen — *Designing Machine Learning Systems* (O'Reilly, 2022): https://www.oreilly.com/library/view/designing-machine-learning/9781098107956/

- Nygard — *Release It!* (2.ª ed., 2018), patrones de estabilidad para la capa de resiliencia: https://pragprog.com/titles/mnee2/release-it-second-edition/

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P117 · AgentBench: evaluar modelos de lenguaje como agentes	2023	Evalúa agentes en ocho entornos distintos y hace visible que la tasa agregada esconde dónde y cómo fallan.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Huyen, Chip — *Designing Machine Learning Systems*	2022	ISBN 9781098107956 · web de la obra · *pendiente de confirmar en su catálogo*	citada en las referencias de esta clase · obra de referencia de la parte 12

Normas y documentación oficial que aplica esta clase: OpenTelemetry Documentation

📝 Evaluación completa

❓ Preguntas

1. Define proyecto: plataforma de ia observable sin usar una marca o framework como definición.
2. Explica la relación entre platform, observability, release, SLO.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.
